

Grado en Ingeniería Informática
Computación

Trabajo de Fin de Grado

**Análisis y mejora en la detección de pose de
humanos en Cesta punta/Pelota Vasca**

Autor/a

Julen Azpiroz Sanchez

2020

Grado en Ingeniería Informática
Computación

Trabajo de Fin de Grado

**Análisis y mejora en la detección de pose de
humanos en Cesta punta/Pelota Vasca**

Autor/a

Julen Azpiroz Sanchez

Directore/a(s)

Roberto Santana Hermida

Resumen

A día de hoy, el campo de *Visión por Computador* está en auge debido a que la disciplina del aprendizaje automático también está evolucionando y como consecuencia, es posible obtener modelos cada vez más complejos y más precisos que son capaces de detectar varias características de una imagen, por ejemplo: dada una imagen, la clase de cada objeto, su cuadro delimitador (también conocido por su término original como “bounding box”), la pose del objeto, etc.

Estos avances, se deben a la evolución de las redes neuronales convolucionales y al uso de técnicas novedosas e ingeniosas que han permitido y están permitiendo lograr unos avances significativos en el mundo de *Visión por computador*. Este proyecto se marca en el estudio de problemas relacionados con la *Visión por computador*. En particular, el problema que se estudiará es el de reconocimiento de poses en imágenes de Pelota vasca y Cesta punta. Con este fin, analizaremos ciertas arquitecturas de redes neuronales convolucionales y a su vez, modelos recientes que hacen uso de estas.

En este trabajo, nos enfocaremos en un modelo reciente, con el objetivo de estudiarlo, comprenderlo y utilizarlo para poder lograr/intentar mejorar el mismo con respecto a imágenes de Pelota Vasca y Cesta Punta. El trabajo busca determinar cómo actúa el modelo para este conjunto de imágenes, y tratar de detectar aquellas posibles imperfecciones para poder mejorarlo, aplicando alguna mejora en el modelo, o bien algún tratado adicional al mismo. Para la implementación, modificación y evaluación del modelo usaremos las librerías “Tensorflow”, “Keras”, alguna propias de “COCO” (hablaremos de esto más adelante) y muchas más, implementadas todas en el lenguaje de programación de Python.

Índice general

Resumen	I
Índice general	III
Índice de figuras	VII
Índice de tablas	XI
1. Introducción	1
1.1. Descripción del trabajo	1
1.2. Contenido de la Memoria	2
2. Gestión del proyecto	5
2.1. Tareas a realizar	5
2.2. Análisis de riesgos	7
2.2.1. Alcance	7
2.2.2. Identificar los riesgos	8
2.2.3. Acontecimiento imprevistos	8
2.3. Planificación inicial del proyecto y su desarrollo	9

3. Redes neuronales convolucionales	13
3.1. Génesis de las redes neuronales convolucionales	13
3.2. Operadores de convolución y agrupamiento	14
3.3. Arquitecturas	16
4. Pre-Investigación realizada	19
4.1. DeepPose: Estimación de pose humana usando redes neuronales profundas	20
4.1.1. Descripción	20
4.1.2. Modelo	21
4.1.3. Resultados del modelo	21
4.2. Localización eficiente de objetos utilizando redes convolucionales	22
4.2.1. Descripción	22
4.2.2. Modelo	22
4.2.3. Resultados del modelo	23
4.3. Máquinas de pose convolucional	24
4.3.1. Descripción	24
4.3.2. Modelo	24
4.4. Estimación de pose humana con retroalimentación de error iterativo	25
4.4.1. Descripción	25
4.4.2. Modelo	26
4.5. Redes de reloj de arena apiladas para la estimación de la postura humana .	27
4.5.1. Descripción	27
4.5.2. Modelo	27
4.5.3. Resultados	28
4.6. Líneas de base simples para la estimación y el seguimiento de la postura humana	29

4.7. Aprendizaje de representación de alta resolución profunda para la estimación de pose humana	30
4.7.1. Descripción	30
4.7.2. Modelo	30
5. Modelo Base: Mask-R-CNN	33
5.1. Descripción y funcionamiento teórico	33
5.1.1. Detección de objeto	34
5.1.2. Segmentación semántica de imagen	35
5.2. Descripción Técnica	35
5.2.1. Faster-R-CNN	35
5.2.2. Segmentación semántica usando FCN	37
5.2.3. Función de error	38
5.3. Arquitectura de la red	38
5.3.1. Red piramidal de características	39
5.3.2. ResNet 101	40
5.4. Estado del arte	42
6. Extensión del modelo a la detección de pose de pelotaris	43
6.1. Detección de poses en Pelota vasca y Cesta punta	43
6.2. Recopilación de imágenes y evaluación del modelo sobre las mismas . . .	44
6.3. Análisis de los resultados del modelo	45
6.3.1. Errores en Pelota vasca	45
6.3.2. Errores en Cesta punta	46
6.4. Mejoras realizadas al modelo	47
6.4.1. Anotación manual del etiquetado del sub-conjunto de imágenes obtenidas	49

6.4.2. Re-entrenamiento del modelo utilizando el subconjunto de imágenes anotado manualmente	49
6.5. Análisis de los resultados	51
6.5.1. Evaluación cuantitativa	51
6.5.2. Análisis visual	54
7. Conclusiones	57
Anexos	
A. Base de datos COCO	61
A.1. Descripción	61
A.2. Etiquetas	61
Bibliografía	67

Índice de figuras

2.1. Planificación inicial del proyecto.	10
2.2. Sucesión resultante de las tareas realizadas.	12
3.1. Número de conexiones en una imagen de 100x100 píxeles.	13
3.2. Aplicación de una convolución en diferentes partes de la imagen.	15
3.3. Proceso de convolución.	15
3.4. Profundidad de capa.	16
3.5. Ejemplos de filtros aprendidos en diferentes objetos.	16
3.6. Ejemplos de operadores de agrupamiento: agrupamiento de media y agrupamiento de máximo.	16
3.7. Esquema de arquitectura AlexNet.	17
4.1. Representación visual de la obtención de un maniquí, Figura obtenida de [Toshev and Szegedy, 2014].	20
4.2. Obtención de articulaciones mediante mapas de calor, Figura obtenida de [Tompson et al., 2015].	22
4.3. Arquitectura en alto nivel del modelo "Localización eficiente de objetos utilizando redes convolucionales". Figura obtenida de [Tompson et al., 2015].	23
4.4. Representación de los diferentes módulos de CPM, Figura obtenida de [Wei et al., 2016].	25
4.5. Arquitectura de la CPM en alto nivel, Figura obtenida de [Wei et al., 2016].	25

4.6. IEF pipeline, Figura obtenida de [Carreira et al., 2016].	26
4.7. Ejemplo del proceso iterativo de IEF para generar una salida, Figura obtenida de [Carreira et al., 2016].	27
4.8. Arquitectura de Relojes de arena apilados en alto nivel, Figura obtenida de [Newell et al., 2016].	28
4.9. Representación visual de un reloj de arena, Figura obtenida de [Newell et al., 2016].	28
4.10. Arquitectura ResNet con deconvoluciones, Figura obtenida de [Xiao et al., 2018].	29
4.11. Arquitectura de HRNET en alto nivel, Figura obtenida de [Sun et al., 2019].	30
5.1. Representación visual de la salida mostrando los maniqués.	34
5.2. Representación visual de la salida mostrando las máscaras.	34
5.3. Segmentación semántica a nivel de píxel para distinguir entre la moto y el conductor.	35
5.4. Pipeline de la red Faster-R-CNN.	36
5.5. Representación visual de ROI Pooling 1/2.	36
5.6. Representación visual de ROI Pooling 2/2.	37
5.7. Visualización de obtención de una máscara sobre una ROI 1/2.	38
5.8. Visualización de obtención de una máscara sobre una ROI 2/2.	38
5.9. Representación en alto nivel de FPN. Figura obtenida de [Lin et al., 2017].	39
5.10. Esquema de como FPN a cada capa obtiene más información de valor semántico pero pierde más resolución. Figura obtenida de ¹	39
5.11. Representación visual de los procesos de ascender y descender en FPN. Figura obtenida de ²	40
5.12. Representación visual de la arquitectura ResNet 101.	41
5.13. Comparativa entre las predicciones realizadas por FCIS (arriba) y Mask-R-CNN (abajo), Figura obtenida de [He et al., 2017].	42
6.1. Ejemplo de predicción correcta sobre Pelota vasca.	45
6.2. Ejemplo de predicción correcta sobre Cesta punta.	45

6.3. Ejemplo del refinamiento impreciso en Pelota Vasca error I.	46
6.4. Ejemplo del refinamiento impreciso en Pelota Vasca error I.	46
6.5. Ejemplo del refinamiento impreciso en Pelota Vasca error II.	46
6.6. Ejemplo del refinamiento impreciso en Pelota Vasca error II.	46
6.7. Ejemplo del refinamiento impreciso en Pelota Vasca error III.	47
6.8. Ejemplo del refinamiento impreciso en Cesta punta 1/2.	47
6.9. Ejemplo del refinamiento impreciso en Cesta punta 2/2.	47
6.10. Ejemplo de la clasificación imprecisa en Cesta punta.	48
6.11. Visualización de 'COCO-annotator' 1/2.	49
6.12. Visualización de 'COCO-annotator' 2/2.	49
6.13. Representación visual de obtención de IoU. Figura obtenida de ³	52
6.14. Ejemplo de mejora del modelo 1/2.	54
6.15. Ejemplo de mejora del modelo 2/2.	54
A.1. Ejemplo visual de las etiquetas de como se representarían de forma orde- nada.	65
A.2. Ejemplo visual de una imagen con la variable 'iscrowd' igual a 1.	65

Índice de tablas

4.1. Comparación de los resultados obtenidos por diferentes variantes de Deep-Pose y otras aproximaciones en la detección de articulaciones.	21
4.2. Tabla de resultados para diferentes partes del cuerpo del modelo.	23
4.3. Tabla de resultados de 'Stacked Hourglass Network'.	29
6.1. Cantidad de personas por imagen.	44
6.2. Estadísticas del modelo antes del re-entrenamiento.	53
6.3. Estadísticas del modelo tras ejecutar el re-entrenamiento.	53
A.1. Información general de la base de datos.	62
A.2. Información de las licencias.	62
A.3. Información de las imágenes.	63
A.4. Información de las anotaciones de cada objeto.	64
A.5. Información de las categorías.	64

1. CAPÍTULO

Introducción

Este documento contiene toda la información en cuanto a investigación, objetivos propuestos, alcanzados, obtenidos, gestión del proyecto, etc. A lo largo de 8 Capítulos donde se pretende que, de forma organizada, se pueda registrar por escrito dicha información.

1.1. Descripción del trabajo

La idea base del trabajo, consistía en realizar un estudio de las redes convolucionales [Girshick, 2015], a la hora de detectar la pose de un objeto en una imagen, con el objetivo de detectar posibles deficiencias y proponer mejoras a las redes. Los modelos son muy interesantes, ya que resolver la tarea de identificar la pose de un objeto, no solo es útil para hacer uso de esa pose, sino que además, permite reconocer mejor los objetos y discernir mejor entre uno y otro. La detección de objetos puede aplicarse, desde reconocimiento de humanos en cámaras de vigilancia en tiempo real, a reconocimiento de objetos válidos o no para tareas hipotéticas de drones o robots, etc. El uso es ilimitado, y está en nuestra imaginación darles utilidad a estos modelos. Conforme el estudio iba avanzando, se decidió tratar la pose de objetos específicamente humanos, ya que acotaba los modelos que se podían analizar así como el volumen de búsqueda/trabajo.

El siguiente paso fue lograr la adquisición de un modelo de red accesible, el cual pudiera ser ejecutado en los ordenadores disponibles, para poder entender su comportamiento y poder profundizar más en el mismo. El modelo escogido fue el de “Keypoints-of-

humanpose-with-Mask-R-CNN”, una extensión del modelo “Mask-R-CNN” [He et al., 2017], del cual hablaremos detalladamente más adelante.

A partir de ese momento, se analizó cuál era el rendimiento del modelo para imágenes de Pelota vasca o de Cesta punta para tratar de percibir algún error o algún comportamiento el cual pudiera ser mejorable. La selección de imágenes de Pelota vasca fue determinada por varias motivaciones: 1) Las poses particulares que se dan en este deporte y los implementos que suelen utilizarse para practicarlo pueden hacer más difícil el proceso de detección de pose. 2) El arraigo local y el interés cultural de este deporte. 3) Y el hecho de que estos modelos no se habrían entrenado con un conjunto de imágenes de este estilo, por lo que estaríamos tratando un terreno virgen. En prueba preliminares comprobamos que efectivamente existen imperfecciones en el comportamiento del modelo, más en imágenes de cesta punta que en imágenes de Pelota vasca.

Finalmente, se optó por realizar una mejora del modelo, tratando de re-entrenar el mismo con un subconjunto de imágenes de Pelota vasca y Cesta punta anotadas manualmente. De esta forma, se logró una mejora leve en el refinamiento de la salida.

1.2. Contenido de la Memoria

Como bien hemos mencionado antes, este documento está estructurado en varios capítulos, con el fin de organizar la información del trabajo además de plantear un orden cronológico de sucesos/trabajos acorde a lo realizado. A continuación describiremos brevemente la función general de cada capítulo.

- (a) Capítulo 1: Se realiza una breve introducción descriptiva del trabajo y de la memoria.
- (b) Capítulo 2: Describe la gestión realizada en el proyecto, las previsiones del mismo tal como estaban pensadas desde un principio y de los cambios que han surgido en el proceso del mismo.
- (c) Capítulo 3: En este capítulo se introducirá el concepto de red neuronal convolucional y por qué son tan eficientes a la hora de trabajar con imágenes. Este capítulo es muy importante para poder comprender el desarrollo del trabajo.
- (d) Capítulo 4: En este capítulo se hablará de los modelos más destacados que se investigaron, los cuales iban progresando paulatinamente en cuanto a complejidad de arquitecturas y uso de las redes convolucionales, así como los resultados obtenidos.

-
- (e) Capítulo 5: En este capítulo se explicará el funcionamiento del modelo utilizado, cómo funciona y cómo actúa a niveles más internos el modelo.
 - (f) Capítulo 6: En este capítulo se realiza una descripción de los problemas detectados en el modelo, las propuestas/alternativas estudiadas y realizadas para resolver dichos problemas así como los resultados de los mismos..
 - (g) Capítulo 7: Por último, en este capítulo se cerrará el documento presentando las conclusiones del proyecto realizado.

2. CAPÍTULO

Gestión del proyecto

2.1. Tareas a realizar

Como ha sido mencionado de forma breve, el trabajo a realizar no estaba fijado desde el principio como “Análisis y mejora en la detección de pose de humanos en Cesta punta/Pelota Vasca” sino más bien como, “Análisis de redes neuronales convolucionales a la hora de detectar la pose de un objeto en una imagen”. Por ello, la idea inicial era realizar una primera investigación no demasiado exhaustiva de una serie de modelos que han ido destacando, no solo por los resultados obtenidos sino también por el uso de arquitecturas de redes neuronales convolucionales, para familiarizarnos con las mismas y así poder fijar el objetivo de este trabajo.

Una vez hecho esto, el siguiente objetivo fue el de lograr acceder a un modelo y una base de datos de fácil comprensión y accesible. El objetivo inicial era estudiar los modelos más significativos de detección de pose para identificar sus limitaciones y, de ser factible, proponer mejoras. Para ello necesitábamos bases de datos específicas y modelos factibles de ser usados, evaluados y modificados en un tiempo razonable. Una vez realizadas las pruebas, identificamos las imágenes de Pelota vasca como un dominio de aplicación interesante por las razones explicadas antes. De esta forma acabamos fijando el objetivo del proyecto como “Análisis y mejora en la detección de pose de humanos en Cesta punta/Pelota Vasca”.

En un principio, antes de alcanzar este estado, se contemplaron varias ideas para el objetivo del trabajo. Se propusieron 3 ideas:

1. Acceder a un modelo ejecutable e implementar alguna “app” o software que hiciera uso de este modelo para su finalidad, es decir, desarrollar alguna especie de aplicación con detección de personas a través de una imagen y que a su vez pudiera tener alguna finalidad adicional.
2. Acceder a un modelo el cual también se pudiera ejecutar e intentar mejorarlo tratando de realizar alguna mejora en su implementación o bien alguna modificación en cuanto a su arquitectura de red neuronal.
3. Realizar el mismo paso anterior, solo que en este caso, tratar de poner a prueba una técnica nueva en dicho modelo, o realizar alguna modificación que no se hubiera realizado y demostrar, que para ese tipo de modelo, dicha técnica **no mejora** el modelo para poder descartar esa idea de mejora en un futuro.

Cuando hablamos de mejora del modelo, nos referimos alternativamente a:

- Menor tiempo de ejecución, ya sea a la hora de ejecutar el modelo o a la hora de re-entrenar el modelo.
- Mayor tasa de acierto en las estadísticas globales de los valores de salida.
- Mayor precisión en la detección de pose, es decir, resultados más refinados o bien resultados representados de una forma más visual.

Finalmente se optó por la opción 2.

Tras alcanzar este punto, la siguiente tarea era experimentar con el modelo haciendo que procesara varias imágenes de Pelota vasca y Cesta punta hasta detectar algún comportamiento extraño o imperfecto en la salida. No hubo que utilizar muchas imágenes, pues no tardamos mucho en descubrir ciertas imperfecciones. En Pelota vasca había algún que otro pequeño error dadas ciertas posturas complejas, pero fue en Cesta punta donde se descubrieron errores más llamativos a partir de los cuales, se podía realizar una mejora.

Finalmente, se realizó la mejora del modelo re-entrenándolo con un subconjunto menor de imágenes. Se anotaron manualmente hasta 100 imágenes de Pelota vasca y Cesta punta (50 imágenes para cada campo), utilizando una herramienta que ofrecían los desarrolladores de la base de datos utilizada. Dicha herramienta, no solo era útil para marcar las articulaciones y definir las mascararas de los humanos, sino que además, dichas anotaciones estaban en el mismo formato que el de la base de datos original, por lo que hacía factible el proceso de re-entrenamiento.

2.2. Análisis de riesgos

En todo proyecto, cuanto mayor sea su volumen de trabajo, más importante es realizar un buen análisis de riesgos de cara a prevenir futuros acontecimientos inesperados, o bien tener una buena estrategia preparada de cara a estos.

2.2.1. Alcance

Objetivos Iniciales

Los objetivos principales son: 1) Investigar el desempeño de los modelos de detección de poses para imágenes de Pelota vasca y Cesta punta 2) Proponer e implementar modificaciones al modelo o modelos para mejorar su desempeño y analizar dicho progreso.

Recursos disponibles

Para la realización de este trabajo se disponía de dos ordenadores, un portátil sencillo sin GPU y un procesador i3 el cual valdría para poder realizar alguna labor de investigación o escritura en cualquier entorno, y un ordenador de sobremesa con una GPU y un procesador i7, con el cual realizar las labores de ejecución de modelos, descargas de bases de datos y a ser posible entrenamientos de modelos. Aunque para esto último es posible que este ordenador no cumpla con las características necesarias, ya que estamos trabajando con modelos recientes, los cuales trabajan con una magnitud de datos cada vez más amplia. Por esta razón, el entrenamiento puede llegar a ser un proceso que dure 14 días.

Ubicaciones, tiempo, inclusiones y exclusiones.

El proyecto se inició en Octubre, en un principio se trató de fijar como objetivo, que fuera finalizado para Febrero, pero no fue el caso, y finalmente se pospuso para el plazo de Junio. Más allá de esto, el proyecto se realizó desde San Sebastián, haciendo uso de las herramientas mencionadas anteriormente, y concertando las reuniones con mi tutor desde la Facultad de Informática o bien vía Skype durante el confinamiento del COVID-19.

2.2.2. Identificar los riesgos

Con respecto a los riesgos con los que se lidió, se identificaron 3 tipos de riesgos fundamentales: el alto coste en tiempo del entrenamiento, limitaciones de hardware o de software a la hora de lograr ejecutar modelos o algún entorno de desarrollo preciso, y lograr entrenar un modelo (en caso de adquirir un modelo eficaz para la tarea planteada), es decir, lograr que se pueda ejecutar el proceso de entrenamiento, más allá de las anteriores limitaciones mencionadas, lograr entender el código de entrenamiento y que se pueda ejecutar y modificar de acuerdo con nuestro interés.

En lo referente al tiempo de entrenamiento, este podría ser el menor de los 3 problemas, ya que como se ha mencionado anteriormente, se contaba con la ventaja de poder ampliar el tiempo hasta Junio si se hubiera considerado necesario. Así que cualquier obstáculo en algún otro problema, implicaría indirectamente más dedicación en tiempo para solucionarlo.

Acerca de los entornos de desarrollo, en un principio se contaba con experiencia en “Anaconda Navigator” y “Jupyter Notebook” para la ejecución de “scripts” de Python que generen, modifiquen y ejecuten modelos, por lo que se espera que pueda darse algún que otro contratiempo, ya que se da por sentado que habrá que lidiar con problemas desconocidos, pero se espera lograr la modificación y ejecución de modelos dentro de un tiempo razonable.

Por último, el mayor obstáculo podría ser re-entrenar un modelo o entrenar desde cero el mismo, ya que no se sabía con qué modelo se iba a trabajar en un principio, ni tampoco sabemos cuánto tiempo puede llegar a durar dicho proceso.

2.2.3. Acontecimiento imprevistos

Pese a las previsiones de riesgos realizadas, naturalmente ha habido ciertos acontecimientos imprevistos. El primero de ellos, la dificultad de entrenar el modelo que se escogió, ya que contaba con una base de datos de 40 gb y el ordenador de sobremesa no era capaz de entrenar debido a las limitaciones de hardware. Los ordenadores que ofrecía la propia Universidad tenían aun mayores limitaciones que el ordenador de trabajo, por lo que la única forma de lograr entrenarlo era tratando de ejecutar los programas en el entorno de desarrollo de “Google colab”, ya que dicho entorno permite hacer uso de un hardware externo ofrecido por Google.

También ha habido percances a la hora de asentar el entorno de trabajo, ya que en un principio, la intención original era trabajar utilizando el sistema operativo de Windows. Se perdió bastante tiempo tratando de ejecutar entornos de desarrollo y modelos con Windows, cuando finalmente se decidió realizar una partición del disco duro en el ordenador de sobremesa para instalar Linux-Mint, fue todo mucho más sencillo. Se hubiera ganado mucho tiempo si se hubiese tomado esa decisión desde el principio.

Además de lo anterior, cabría mencionar la aparición de la epidemia COVID-19, y sus repercusiones en este proyecto. Por suerte se ha podido realizar la gran mayoría de las labores desde el hogar, tan solo ha afectado a la posibilidad de no asistir a la biblioteca y otros entornos de trabajo similares, así como no poder concertar reuniones presenciales con el tutor del proyecto, para lo cual, sencillamente se ha podido realizar vía Skype.

2.3. Planificación inicial del proyecto y su desarrollo

La planificación ha variado desde que se realizó un primer esquema inicial, como se ha mencionado anteriormente, la idea inicial era finalizarlo para Febrero, y sin haber determinado cuáles serían las labores concretas, más allá de los objetivos propuestos, el primer diagrama resultante fue el que se muestra en la Figura 2.1:

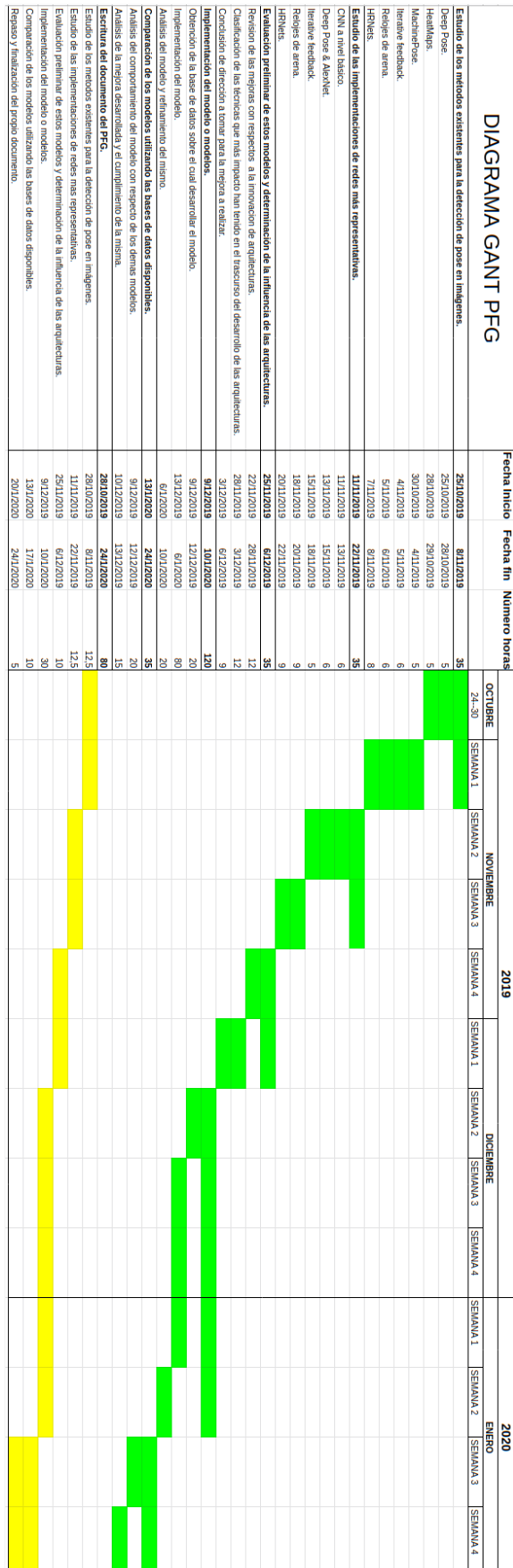


Figura 2.1: Planificación inicial del proyecto.

Como se puede apreciar en la Figura 2.1, en un principio se pretendió estudiar las metodologías de las diferentes arquitecturas revolucionarias que fueron surgiendo y después estudiar sus métodos de implementación. Esto se cumplió y fue la primera fase de lo que en realidad fue la labor del proyecto. No se estudiaron a fondo los diferentes modelos, simplemente se trataron de entender cuales fueron sus aspectos más característicos y por qué fueron fundamentales en la historia de la evolución de las redes neuronales convolucionales concebidas para la detección de pose.

Una vez concluida esa fase, se decidió que se iba realizar una evaluación preliminar de estas arquitecturas para decidir cuál iba a ser escogida como mejora, pero antes de ejecutar este paso, hubo una tarea no planificada, la adquisición de un modelo accesible junto a una base de datos. Este paso que un principio se considero simple de implementar, resultó ser más complicado de lo previsto y desvirtuó la dirección de la planificación inicial por completo.

A continuación se describe la tarea que sería la parte complicada del proyecto, la implementación de la mejora en el modelo adquirido. En el caso del resultado final no difiere mucho, ya que la parte más complicada de éste ha sido la mejora del modelo también, salvo que en el caso resultante ha sido más una tarea de investigación y aprendizaje que de implementación.

El resultado de lo que fue el orden de acontecimientos exactos, se describe en la Figura 2.2:

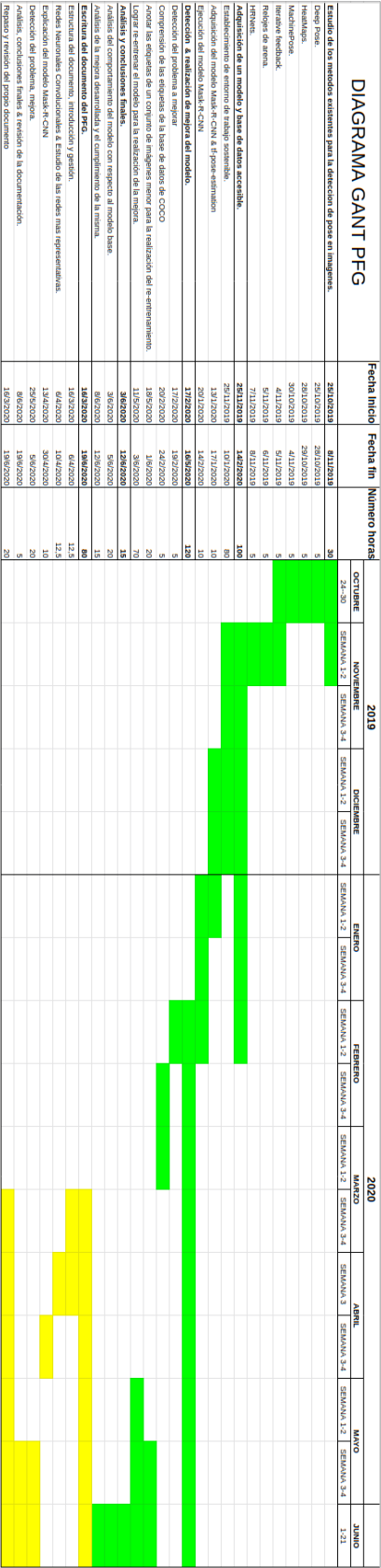


Figura 2.2: Sucesión resultante de las tareas realizadas.

3. CAPÍTULO

Redes neuronales convolucionales

3.1. Génesis de las redes neuronales convolucionales

Las redes neuronales convolucionales nacen con la necesidad de poder procesar imágenes de una manera eficiente. Hoy en día también se utilizan para procesamiento de texto, pero su razón de existir es para procesamiento de imágenes, ya sea en fotografías estáticas o vídeos. Pero, ¿por qué son tan efectivas? Supongamos que queremos procesar una imagen cualquiera, la cual contiene 100 píxeles de ancho, 100 píxeles de altura y 3 canales de colores, estamos hablando de una imagen de muy baja resolución, ya que hoy en día las imágenes más comunes suelen tener 1920x1080 o 1366x768 píxeles. Esta imagen, pese a su baja resolución, si es introducida como entrada a una red multi-capa, cuya primera capa oculta contiene 100 neuronas, necesitaría $100(\text{neuronas}) \times 100(\text{píxeles de altura}) \times 100(\text{píxeles de anchura}) \times 3(\text{canales de colores}) = 3.000.000$ de conexiones en la primera capa.

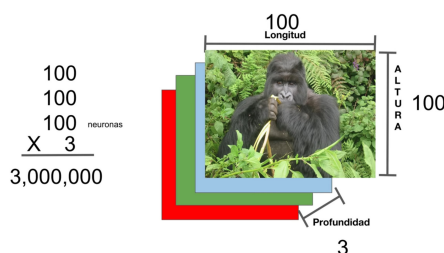


Figura 3.1: Número de conexiones en una imagen de 100x100 píxeles.

Esta metodología de procesar una imagen píxel por píxel es muy ineficiente. Si trabajásemos con una imagen de 1920x1080 el tiempo de procesamiento se elevaría notablemente. Las redes convolucionales aplican una nueva metodología a la hora de procesar imágenes, en vez de procesar una imagen píxel por píxel, lo que hacen es aplicar *convoluciones*, las cuales detectan patrones o características de una imagen, y también aplican la técnica de *agrupamiento* (también conocida por su termino original como “pooling”), técnica que generalmente se utiliza para reducir el tamaño espacial de una región de la imagen, permitiendo reducir el número de parámetros manteniendo la información relevante de dicha región. Gracias a estas dos técnicas y junto a otras operaciones menores, el número de conexiones necesarias por la red convolucional es mucho menor que en la red multi-capas, por ello se logra reducir considerablemente el volumen de computación a la hora de procesar una imagen.

Además de lo anterior, cuando hablamos de procesar una imagen, generalmente hablamos de “tratar de entender” la imagen, es decir, en el caso del gorila de la Figura 3.1, si queremos identificar dónde está el gorila y reconocer que es un gorila en lugar de otra clase de objeto, tenemos que aprender a contextualizar la imagen y saber qué partes de la imagen son relevantes y cuales no. Por esta razón las *convoluciones* no solo son útiles para reducir el número de conexiones, sino también para realizar un análisis más “inteligente” de las imágenes.

3.2. Operadores de convolución y agrupamiento

Las *convoluciones* se aplican recorriendo la imagen completa, tomando en cada paso un píxel como referencia, y realizando la *convolución* entre el filtro o parche, que es una matriz de valores numéricos, y el conjunto de píxeles en torno al píxel de referencia escogido. El operador de *convolución* se aplica a cada capa de color y lo que trata de obtener son diferentes patrones o características de una imagen, estos pueden ser bordes, sombras, relieves etc. Véase la Figura 3.2 que ilustra lo que sería el recorrido de una *convolución* a través de una imagen.

Pero, ¿cómo obtiene estas características una *convolución*? Las capas ocultas convolucionales están compuestas de filtros, estos filtros generalmente serán matrices con valores inicializados con valores aleatorios, los cuales, durante el proceso de entrenamiento, se irán ajustando a unos valores que permitirán obtener las características relevantes. Véase la Figura 3.3 .

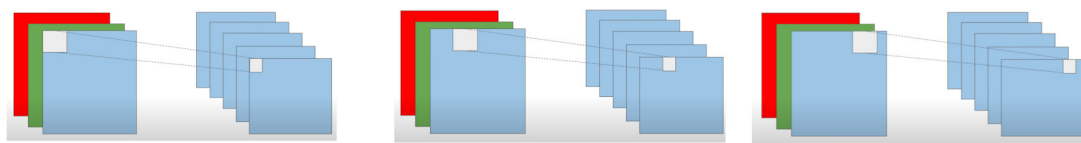


Figura 3.2: Aplicación de una convolución en diferentes partes de la imagen.

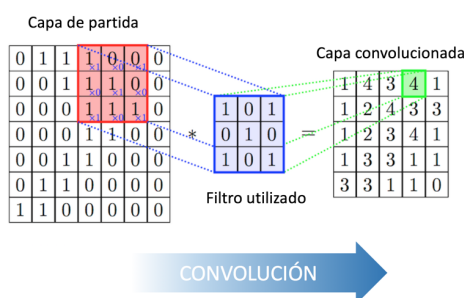


Figura 3.3: Proceso de convolución.

En el proceso de *convolución* hay 3 aspectos fundamentales que dan variedad al tipo de técnica que deseemos aplicar, estos son: el tamaño del filtro, el “stride” o *desplazamiento*, y la profundidad de la capa. El tamaño del filtro definiría el tamaño de la agrupación de píxeles que se obtiene. En el ejemplo de la Figura 3.3 estaríamos operando con un filtro de 3x3, pero las dimensiones de los filtros pueden ajustarse a diferentes criterios de diseño de la red convolucional; el “stride” o *desplazamiento* representa el número de saltos de píxeles entre filtro y filtro. El stride puede utilizarse para reducir el número de parámetros de la red. Esto podría agilizar los procesos de *convolución*. Por último, la profundidad de capa es el número de filtros que obtenemos por cada proceso de *convolución*, generalmente las imágenes tienen 3 profundidades, que son los esquemas de color rojo, verde y azul (a menos que se trate de una imagen en blanco y negro). Por otro lado, al realizar una *convolución*, siempre se acotará su tamaño en altura y anchura pero siempre obtendremos una mayor profundidad.

Tras aplicar el operador de *convolución*, no solo se reduce considerablemente el número de conexiones, sino que también se obtienen características de una imagen, tal y como se puede apreciar en la Figura 3.5, dichas características hacen que sea más fácil reconocer patrones como bordes, sombras, relieves, etc. Este hecho facilita el reconocimiento de objetos, y muchas otras finalidades.

La operación de *agrupamiento* (pooling) se aplica de manera similar a una *convolución*, pero en cada aplicación a un filtro de píxeles se obtiene únicamente un valor resultante.

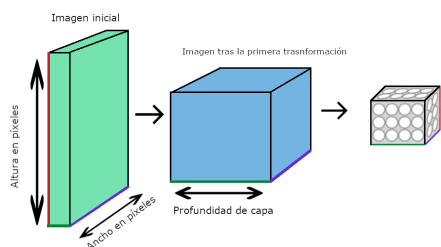


Figura 3.4: Profundidad de capa.

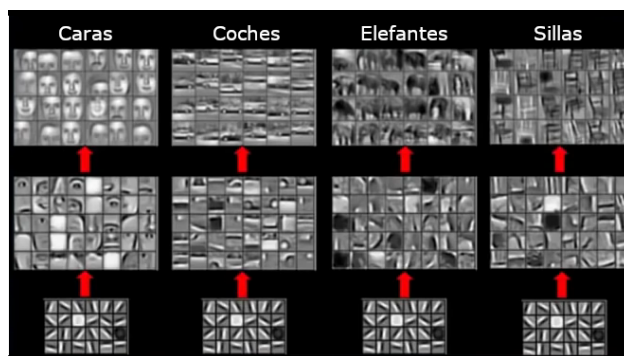


Figura 3.5: Ejemplos de filtros aprendidos en diferentes objetos.

De esta manera se reduce el tamaño de la imagen. Existen dos tipos de *agrupamiento*: *agrupamiento de media*, en el cual se obtiene el valor final como resultado de una media de los valores de los píxeles, y *agrupamiento de máximo*, donde el valor final se obtiene como resultado de tomar el valor máximo de los píxeles.

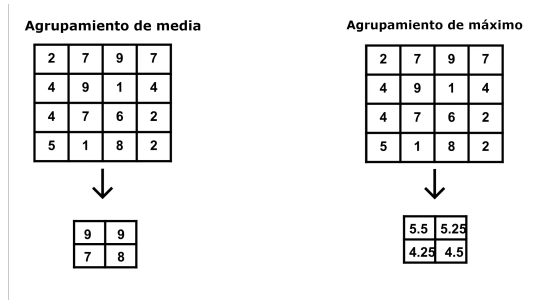


Figura 3.6: Ejemplos de operadores de agrupamiento: agrupamiento de media y agrupamiento de máximo.

3.3. Arquitecturas

Con la combinación de las operaciones *convolución* y de *agrupamiento* es de donde surgen las primeras redes neuronales convolucionales. Una de las arquitecturas más conocidas es la denominada “AlexNet” [Alom et al., 2018], la cual consiste en una sucesión de 8 capas, las primeras 5 de ellas son *convoluciones* y las 3 últimas son multicapa feedforward (capa completamente conectada). Generalmente en las 2 primeras capas de *convolución* y la última, después de el proceso de *convolución* se introduce una operación de *agrupamiento de máximo*.

El esquema sería el siguiente Conv1 + MP -> Conv2 + MP -> Conv3 -> Conv4 -> Conv5

+ MP ->FC1 ->FC2. Aunque a la estructura original de AlexNet se le han introducido numerosas variaciones, su arquitectura se podría describir como la mostrada en la Figura 3.7, que es solamente un ejemplo entre diferentes variantes de esta arquitectura.

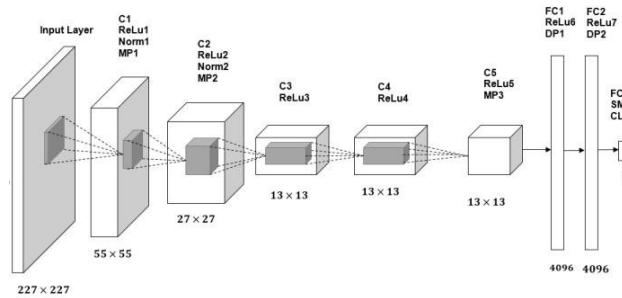


Figura 3.7: Esquema de arquitectura AlexNet.

Gracias a los diferentes usos de *convoluciones*, *agrupamiento*, combinación de arquitecturas y otras ideas más creativas, han ido surgiendo los diferentes modelos que han permitido la evolución del mundo del aprendizaje automático dentro del campo de visión por computador.

Es por ello, que el estudio de los modelos de redes convolucionales más destacados fue el primer paso para este proyecto, para una vez comprendidos los principales modelos y sus características, poder identificar un modelo apropiado para nuestro trabajo y proceder a realizar una mejora del mismo.

4. CAPÍTULO

Pre-Investigación realizada

En este capítulo analizaremos los modelos más destacados en el uso de redes neuronales convolucionales con el objetivo de detectar la pose de un humano en una imagen. El orden de presentación de modelos que hemos seguido es cronológico, comenzando con los más antiguos y terminando con los más recientes.

La intención del análisis no es profundizar en cada uno de los modelos pues requeriría mucho tiempo, pero sí resulta necesario un análisis sobre las principales características de estos modelos, las distintas técnicas, basamento teórico, funcionamiento práctico, ver cuáles fueron las distintas técnicas de representar la salida, diferentes tipos de arquitecturas, etc.

Se estudiaron los siguientes modelos:

- DeepPose: Estimación de pose humana usando redes neuronales profundas.
- Localización eficiente de objetos utilizando redes convolucionales.
- Máquinas de pose convolucional.
- Estimación de pose humana con retroalimentación de error iterativo.
- Redes de reloj de arena apiladas para la estimación de la postura humana.
- Líneas de base simples para la estimación y el seguimiento de la postura humana.
- Aprendizaje de representación de alta resolución profunda para la estimación de pose humana.

4.1. DeepPose: Estimación de pose humana usando redes neuronales profundas

4.1.1. Descripción

DeepPose [Toshev and Szegedy, 2014], fue el primer modelo en lograr predecir la pose de un humano en una imagen de forma significativa, gracias a la aplicación del aprendizaje profundo y el uso de las RNC (Redes neuronales convolucionales). Se trata de un modelo de interés pues además de lograr superar a todos los modelos anteriores, fue capaz de obtener la pose de un humano de manera holística, es decir, que incluso si una parte del cuerpo no se encuentra visible, ya sea porque se encuentra oculto tras un objeto o debido a alguna otra razón, el modelo genera las coordenadas de dicha parte.

La salida del modelo trata de devolver para cada humano donde se encuentran sus articulaciones en la imagen, cada articulación se representa con dos valores X e Y que muestran la coordenada en píxel donde se encuentra dicha articulación. Para cada persona, se obtienen las coordenadas X_i e Y_i , de $i \in \{1, \dots, K\}$, y K es el número de articulaciones en la imagen. Una vez obtiene todas las coordenadas de las articulaciones, realiza una unión de las mismas y de esta forma genera un maniquí.

$$(x_i, y_i) * 2 \text{ for } i \in \{1, 2 \dots k\} \quad (4.1)$$



Figura 4.1: Representación visual de la obtención de un maniquí, Figura obtenida de [Toshev and Szegedy, 2014].

4.1.2. Modelo

La arquitectura de este modelo está basada en una red AlexNet, con la ligera modificación de contar con 7 capas en lugar de 8. La salida genera las coordenadas (X,Y), de cada articulación, y después hace una unión de estas para generar un maniquí. El modelo hace uso de la función de error L2, también conocida como función de errores mínimos cuadrados. Véase la Ecuación 4.2

$$L2 = \sum_{i=1}^n (Y_{true} - Y_{predicted})^2 \quad (4.2)$$

Lo interesante también de este modelo es el refinamiento que va logrando a cada capa, es decir, desde la imagen inicial, se va recortando ésta a cada capa procesada, logrando imágenes con mayor resolución para menos espacio, que a su vez permiten obtener características más detalladas de los alrededores de cada articulación.

4.1.3. Resultados del modelo

Para evaluar la calidad de los resultados se utiliza Porcentaje de partes correctas (PCP): Una extremidad se considera detectada (una parte correcta) si la distancia entre las dos ubicaciones previstas de la articulación y las ubicaciones verdaderas de la articulación de la extremidad es inferior a la mitad de la longitud de la extremidad (comúnmente denominada PCP@0.5).

Método	Brazos		Piernas		Media
	Mejor tasa	Peor Tasa	Mejora tasa	Peor tasa	
DeepPose-st1	0.5	0.27	0.74	0.65	0.54
DeepPose-st2	0.56	0.36	0.78	0.70	0.60
DeepPose-st3	0.56	0.38	0.77	0.71	0.61

Tabla 4.1: Comparación de los resultados obtenidos por diferentes variantes de DeepPose y otras aproximaciones en la detección de articulaciones.

Pese a que el modelo fue revolucionario, sus resultados tampoco son apabullantes, la Tabla 4.1 muestra las tasas de acierto utilizando diferentes variantes de DeepPose. En la

detección de los brazos se alcanza una tasa de acierto máxima de **0.56** mientras que para las piernas se obtiene un máximo de **0.78**.

4.2. Localización eficiente de objetos utilizando redes convolucionales

4.2.1. Descripción

El modelo introducido en [Tompson et al., 2015] se caracteriza porque en vez de obtener las articulaciones de forma directa, genera mapas de calor para cada articulación en paralelo. Un mapa de calor predice la probabilidad de que se produzca la unión en cada píxel. Este modelo de salida tuvo mucho éxito y numerosos modelos que le sucedieron tomaron como base esta técnica.

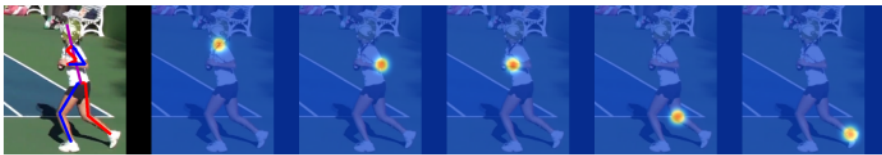


Figura 4.2: Obtención de articulaciones mediante mapas de calor, Figura obtenida de [Tompson et al., 2015].

4.2.2. Modelo

Se utilizan dos arquitecturas de RNC, una (mapa de calor grueso) para obtener el conjunto de ventanas de calor, tal y como se puede visualizar en la Figura 4.2, y la otra red neuronal (mapa de calor fino) se usa para refinar la localización exacta de estas articulaciones y combinarla con la primera, para después generar el mapa de calor más preciso. Sin embargo, a diferencia de las arquitecturas en cascada clásicas como las de AlexNet, esta combinación de arquitecturas, reutiliza características obtenidas de las convoluciones. De esta forma no solo se reduce el número de conexiones, sino que también se produce un efecto regularizador para el modelo de mapa de calor grueso, ya que los modelos grueso y fino se entrenan conjuntamente.

El modelo de mapa de calor fino es un modelo gráfico, mientras que el grueso es una RNC.

Los modelos gráficos aprenden a contextualizar las características del entorno entre dos articulaciones.

El modelo se entrena utilizando la función del error cuadrático medio, calculando la diferencia espacial entre la zona de calor predicha de la articulación correspondiente y el mapa de calor real de dicha articulación.

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_{true} - Y_{predicted})^2 \quad (4.3)$$

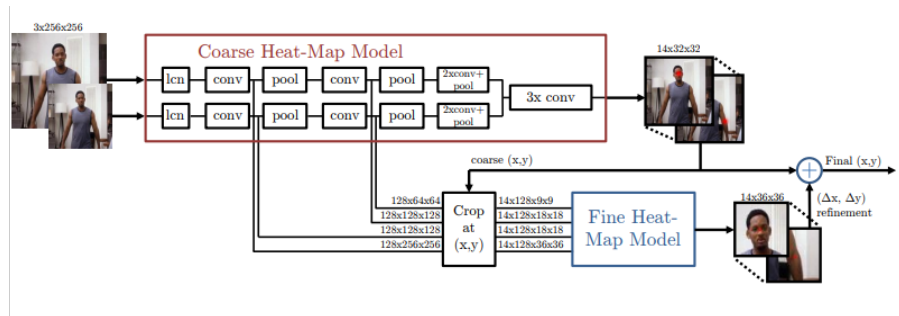


Figura 4.3: Arquitectura en alto nivel del modelo "Localización eficiente de objetos utilizando redes convolucionales". Figura obtenida de [Tompson et al., 2015].

4.2.3. Resultados del modelo

	Cabeza	Hombros	Codos	Muñecas	Caderas	Rodillas	Tobillos	Cuerpo superior	Cuerpo completo
Escala normalizada	96.1	91.9	83.9	77.8	80.9	72.3	64.8	84.5	82.0
Escala normalizada (Solo con el conjunto de prueba)	93.5	87.5	75.5	67.8	68.3	60.3	51.7	77.0	73.3
Escala no normalizada	83.4	77.5	67.5	59.8	64.6	55.6	46.1	68.3	66.0

Tabla 4.2: Tabla de resultados para diferentes partes del cuerpo del modelo.

Como se puede apreciar en la Tabla 4.2, los resultados son algo mejores que los obtenidos con el modelo DeepPose. En este caso obtenemos una información más precisa,

pues ahora se cuantifican las tasas de acierto de otras articulaciones como muñecas, caderas, tobillos, etc. Hay una innovación en la técnica aplicada que se ve reflejada en los resultados.

Se puede apreciar que los dos anteriores modelos descritos en las secciones 4.1 y 4.2, no hacen uso de la ventaja que supone conocer la estructura del cuerpo humano, como por ejemplo la existencia de la simetría, que los codos no puedan doblarse hacia atrás y las rodillas hacia delante, etc. Este tipo de información podría usarse a favor de la predicción como modelo de estructura. Los siguientes trabajos consideran este tipo de información.

4.3. Máquinas de pose convolucional

4.3.1. Descripción

Es relevante analizar el trabajo de [Wei et al., 2016] como hace uso de un concepto inédito llamado *máquinas de pose* (de ahí el nombre del artículo) que denominaremos CPM (convolutional pose machine). Una CPM consiste en un módulo de cálculo de características de una imagen seguido de un módulo de predicción. Las *máquinas de pose* son completamente diferenciables y su arquitectura se puede entrenar de extremo a extremo. El aprendizaje de extremo a extremo generalmente se refiere a omitir cualquier algoritmo intermedio implementado manualmente y permitir a la red aprender directamente la solución de un problema dado el conjunto de datos presentado como base de datos. La ventaja que esto supone es que puede implicar la concatenación de diferentes redes que se entrenan simultáneamente.

Una de las principales motivaciones de este artículo fue aprender las relaciones espaciales de largo alcance y demostrar que esto se puede lograr mediante el uso de campos receptivos más grandes.

4.3.2. Modelo

En la Figura 4.4 se puede apreciar que $g1()$ y $g2()$ obtienen los ya conocidos mapas de calor. La etapa 1 incluiría el módulo de cálculo de características de la imagen, y la etapa 2 estaría definida por el módulo de predicción. La Figura 4.5 representaría la arquitectura a más alto nivel con mayor detalle.

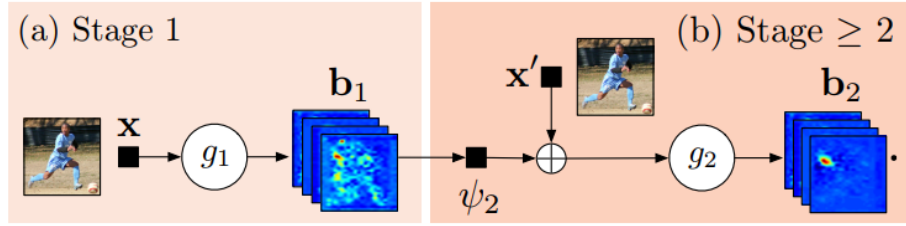


Figura 4.4: Representación de los diferentes módulos de CPM, Figura obtenida de [Wei et al., 2016].

Las CPM constan como mínimo de 2 etapas, según el documento descriptivo generalmente de 3. La etapa 1 es fija, y las etapas siguientes son repeticiones de la etapa 2 que van de 2 a n . Los mapas de calor agregan contexto espacial a las etapas posteriores, por ello lo que se hace en la etapa 1 es obtener el mapa de calor, y en las siguientes etapas se van refinando esos mapas de calor.

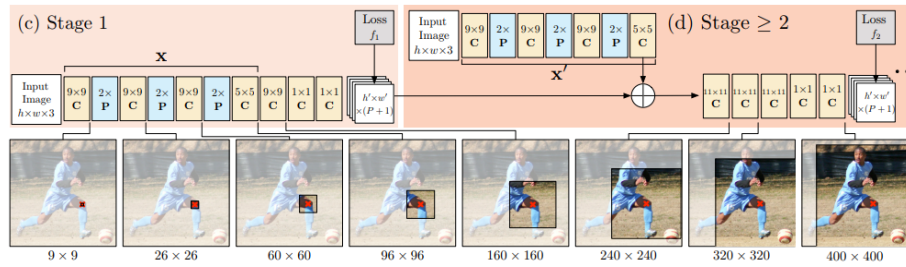


Figura 4.5: Arquitectura de la CPM en alto nivel, Figura obtenida de [Wei et al., 2016].

4.4. Estimación de pose humana con retroalimentación de error iterativo

4.4.1. Descripción

Este artículo [Carreira et al., 2016] resulta bastante interesante. La idea propuesta en el artículo es que, en lugar de tratar de predecir las articulaciones en una única salida, se empieza generando un maniquí en una pose normal e iterativamente ir trasladando las articulaciones a donde deberían estar. El método utiliza un modelo de auto corrección que iterativamente se va corrigiendo hasta obtener la salida correspondiente, de ahí el nombre de “Iterative Error FeedBack” (IEF). En la Figura 4.7 se ilustran dos ejemplos de este proceso.

4.4.2. Modelo

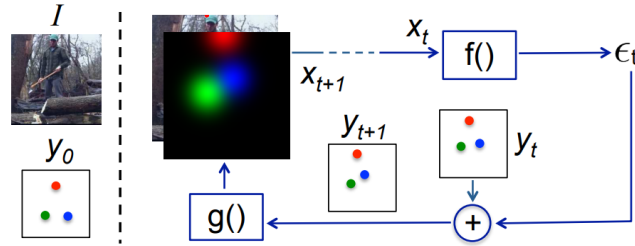


Figura 4.6: IEF pipeline, Figura obtenida de [Carreira et al., 2016].

Sea la imagen original I que se utiliza como entrada, y la representación de la salida anterior Y_{t-1} (la cual va a ser refinado por el algoritmo). Cada entrada X_t se calcularía como:

$$X_t = I \oplus g(Y_t - 1) \quad (4.4)$$

La función $f(x_t)$, generaría una nueva salida, la cual se añadiría como actual salida a Y_t para generar Y_{t+1} . Que tiene la corrección en cuenta.

$$\begin{aligned} \epsilon_t &= f(X_t) \\ Y_{t+1} &= Y_t + \epsilon_t \\ X_{t+1} &= I \oplus g(Y_{t+1}) \end{aligned} \quad (4.5)$$

$g(Y_{t+1})$ convierte cada articulación (Y_{t+1}) en un mapa de calor para que puedan apilarse en la imagen I , a fin de formar la entrada para la próxima iteración. Este proceso se repite T veces hasta obtener un valor de salida Y_{t+1} refinado el cual se lo suficientemente preciso como para considerarse una solución valida.

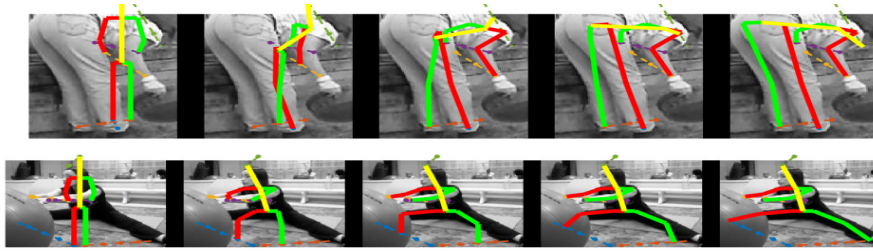


Figura 4.7: Ejemplo del proceso iterativo de IEF para generar una salida, Figura obtenida de [Carreira et al., 2016].

4.5. Redes de reloj de arena apiladas para la estimación de la postura humana

4.5.1. Descripción

Este artículo [Newell et al., 2016] fue otro de los grandes puntos de inflexión en la investigación de arquitecturas novedosas para detección de pose. A esta aproximación se le conoce como relojes de arena apilados, ya que la arquitectura de la red tiene una forma de varios relojes de arena concatenados entre sí. El diseño del reloj de arena surge con la necesidad de capturar información en todas las escalas, ya que hay veces que es importante obtener información a baja escala, para capturar características de las manos, o para los pies, etc. Pero para la estimación de la pose final se requiere un contexto global. La orientación de la persona, la disposición de sus extremidades y las relaciones de las articulaciones adyacentes, se encuentran entre las muchas señales que se reconocen mejor a diferentes escalas en la imagen.

4.5.2. Modelo

La red realiza un procesamiento repetido ascendente y descendente con supervisión intermedia.

- Procesamiento descendente (desde resoluciones altas hasta resoluciones bajas).
- Procesamiento ascendente (desde resoluciones bajas hasta resoluciones altas).

La red utiliza saltos de conexiones que tienen como objetivo preservar la información espacial obtenida en cada resolución, es decir, antes de reducir la resolución de la imagen en

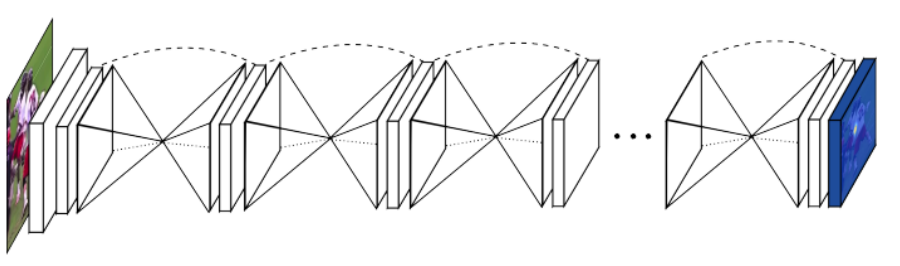


Figura 4.8: Arquitectura de Relojes de arena apilados en alto nivel, Figura obtenida de [Newell et al., 2016].

el nivel X que nos encontremos, las características que puedan ser interesantes o relevantes se pasan al siguiente nivel tal como están antes de reducir la resolución actual, y así no perder la información espacial que se podría perder al contar con menos resolución.

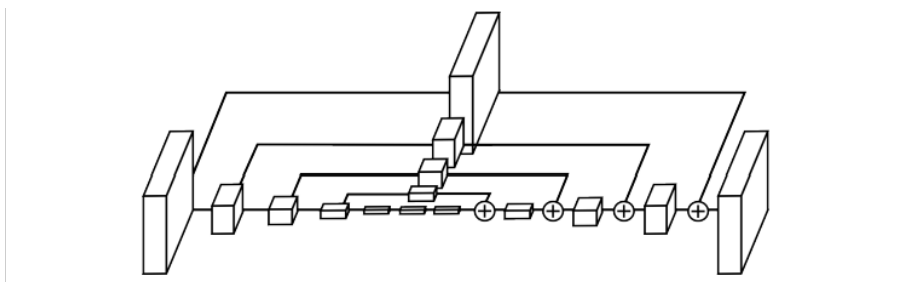


Figura 4.9: Representación visual de un reloj de arena, Figura obtenida de [Newell et al., 2016].

Además, cada reloj de arena contiene supervisiones intermedias. Las supervisiones intermedias se aplican a las predicciones de cada etapa del reloj de arena, es decir, se supervisan las predicciones de cada reloj de arena en la pila, y no solo las predicciones finales del reloj de arena.

4.5.3. Resultados

Como se puede apreciar en la Tabla 4.3 los resultados del modelo propuesto en [Newell et al., 2016] son muy competitivos alcanzando un **98 %** en las predicciones de la cabeza. Si comparamos las tasas obtenidas en este modelo con los mostrados en el modelo 4.2 en la Tabla 4.2 hay una mejora considerable, sobretodo si observamos las tasas de las muñecas y tobillos siendo la diferencia de casi un **20 %** en esta última.

Esta arquitectura permite obtener información espacial a nivel global y a nivel local, lo que permite dar un salto evolutivo en los modelos de predicción de pose.

	Cabeza	Hombros	Codos	Muñecas	Caderas	Rodillas	Tobillos	Total
Modelo	98.2	96.3	91.2	87.1	90.1	87.4	83.6	90.9

Tabla 4.3: Tabla de resultados de 'Stacked Hourglass Network'.

4.6. Líneas de base simples para la estimación y el seguimiento de la postura humana

Este artículo [Xiao et al., 2018] resultó relevante porque a diferencia de los anteriores modelos que se iban volviendo cada vez mas complejos técnicamente, trató de demostrar qué tal rendiría un modelo en este momento, utilizando métodos más simples, aplicando lo aprendido hasta ahora.

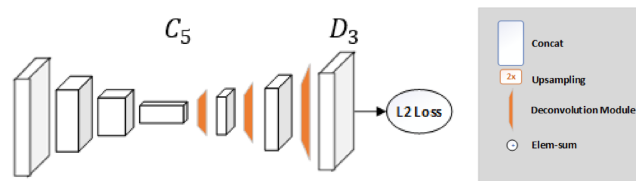


Figura 4.10: Arquitectura ResNet con deconvoluciones, Figura obtenida de [Xiao et al., 2018].

La estructura de la red es bastante simple, consiste en una ResNet con la extensión de unas capas deconvolucionales al final, siguiendo la idea de un reloj de arena. Si bien la red de reloj de arena utiliza el muestreo ascendente para aumentar la resolución del mapa de características y coloca los parámetros convolucionales en otros bloques, este método los combina como capas deconvolucionales de una manera muy simple. Resulta relevante comprobar que una arquitectura con métodos mas simples funciona mejor que una con saltos de conexiones que conservan la información para cada resolución.

Se utiliza la función del error cuadrático medio 4.3 entre el mapa de calor predicho y los mapas de calor objetivos. El mapa de calor objetivo H^K para K articulaciones es generado aplicando una gaussiana 2D a cada área real de las articulaciones con una desviación estándar de 1 píxel.

Cabe destacar que pese a su simpleza éste método obtuvo un resultado del 73,7% en el estado del arte de la base de datos de COCO [Veit et al., 2016], base de datos sobre la que trabaja el modelo adquirido para este proyecto.

4.7. Aprendizaje de representación de alta resolución profunda para la estimación de pose humana

4.7.1. Descripción

El modelo HRNet [Sun et al., 2019] (Red de alta resolución) ha superado a todos los métodos anteriores en tareas de detección de puntos clave o articulaciones, estimación de pose de múltiples personas y estimación de pose de los objetos del conjunto de datos COCO, y es el modelo más reciente. HRNet sigue una idea muy simple. La mayoría de los trabajos anteriores partieron de una representación de alta resolución a baja resolución para después volver a alta resolución sucesivamente. La idea característica de HRNet es mantener durante todo el proceso la alta resolución para no perder la información y cabe advertir que su comportamiento es muy eficiente.

4.7.2. Modelo

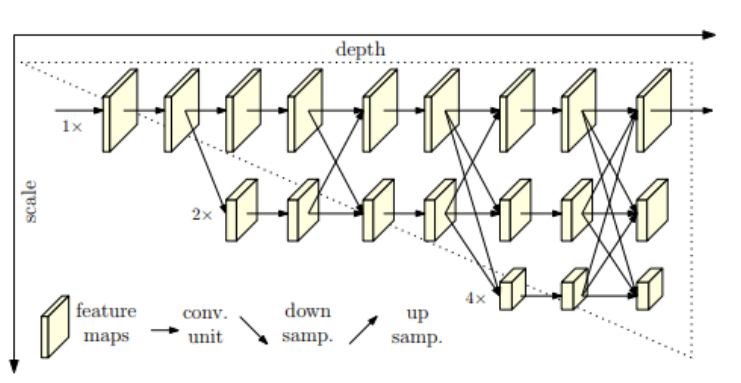


Figura 4.11: Arquitectura de HRNET en alto nivel, Figura obtenida de [Sun et al., 2019].

La arquitectura comienza desde una subred de alta resolución como la primera etapa, y progresivamente agrega subredes de alta a baja resolución una por una para formar más etapas y conectar las subredes de resolución múltiple en paralelo tal y como se puede apreciar en la Figura 4.11. Las fusiones repetidas a varias escalas se realizan mediante el intercambio de información a través de subredes paralelas de resolución múltiple una y otra vez durante todo el proceso. Otra ventaja a tener en cuenta es que esta arquitectura no utiliza la supervisión intermedia del mapa de calor, a diferencia del reloj de arena apilado.

Los mapas de calor se refinan usando la función MSE (Ecuación 4.3), similar al modelo de la sección “Líneas de base simples” 4.6.

5. CAPÍTULO

Modelo Base: Mask-R-CNN

El modelo adquirido fue el de “Keypoints-of-humanpose-with-Mask-R-CNN”, el cual es una extensión del modelo “Mask-R-CNN” [He et al., 2017]. El modelo original es capaz de predecir la pose de una gran cantidad de objetos, sin embargo, la extensión que tratamos se centra solo en obtener la pose de los humanos, para obtener resultados más refinados sobre humanos. En este capítulo se presentará en detalle cómo funciona el modelo original, ya que dicha extensión no contiene un artículo donde se presente su descripción.

Este modelo se utilizó por dos razones: primero, porque era un modelo reciente que tenía buenos resultados; y segundo y lo más importante, fue un modelo acreditado el cual, fue capaz de ejecutarse en el entorno disponible. Hubo otros modelos adquiridos que tenían varios problemas de librerías requeridas, versiones anteriores a otras librerías que traían otro tipo de incompatibilidades, etc. Por ello, una vez se logró ejecutar este modelo, se decidió proseguir con el proyecto y tratar de realizar la mejora sobre este modelo.

5.1. Descripción y funcionamiento teórico

Mask-R-CNN (Masked Region based Convolutional neural network) es un modelo que emplea la fusión de dos arquitecturas, y cada arquitectura ejecuta una tarea específica. Las arquitecturas que incorpora son Faster-R-CNN [Ren et al., 2015] (un modelo previo) y FCN [Long et al., 2015] (fully convolutional network).

A nivel teórico, el modelo Mask-R-CNN tiene dos objetivos: realizar la detección del ob-

jeto, y después realizar una segmentación semántica de imagen para diferenciar los objetos. La salida se representa de dos formas: primero genera los maniquíes correspondientes para cada cuadro delimitador predicho, segundo, se utiliza una máscara de un color para representar la superficie del humano en la imagen. Si aparece más de un humano en una imagen, se utilizan diferentes colores para cada máscara. Las figuras 5.1 y 5.2 ilustran la representación de la salida tanto del maniquí como de la máscara.

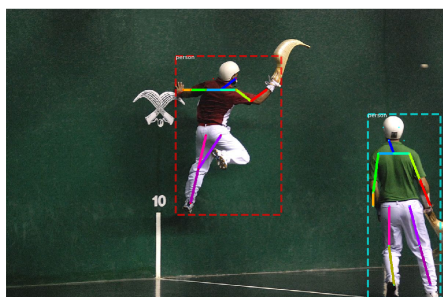


Figura 5.1: Representación visual de la salida mostrando los maniquíes.

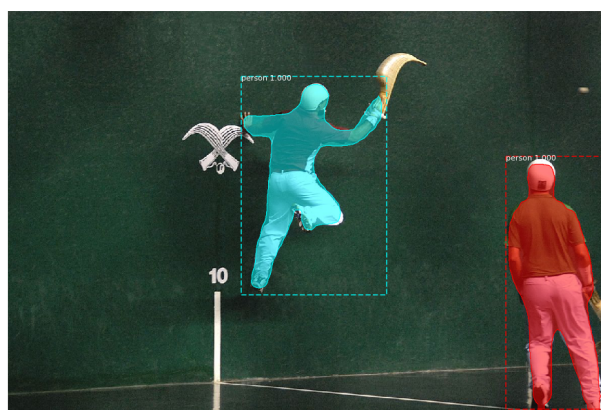


Figura 5.2: Representación visual de la salida mostrando las máscaras.

5.1.1. Detección de objeto

La detección de objeto consiste en identificar los diferentes objetos en una imagen, siempre y cuando el objeto en cuestión esté registrado, es decir, el modelo contiene una base de datos extensa que clasifica una gran variedad de objetos, desde coches, personas, semáforos... hasta objetos menores como camisetas, zanahorias, botellas, etc. Por ello, es conveniente aclarar que no todos los objetos serán reconocidos, pues no todos están registrados.

Para la detección de objetos, primero se trata de predecir el cuadro delimitador de cada objeto, y después de este proceso, para cada cuadro delimitador, se predice la clase de ese objeto. Es decir, primero el modelo reconoce dónde están los objetos, y después, qué objetos son aquellos que han sido reconocidos.

5.1.2. Segmentación semántica de imagen

La segmentación semántica consiste en la tarea de diferenciar los objetos entre sí a nivel de píxel, distinguir qué píxeles pertenecen a un objeto, y cuáles no. Véase el ejemplo que se muestra en la Figura 5.3 como resultaría distinguir a nivel de píxel entre la moto y el conductor.



Figura 5.3: Segmentación semántica a nivel de píxel para distinguir entre la moto y el conductor.

La combinación de aplicar las técnicas de detección de objetos y segmentación semántica dan lugar al concepto de segmentación de una instancia (instance segmentation).

5.2. Descripción Técnica

Como se ha mencionado anteriormente, este modelo está basado en una mezcla de dos arquitecturas. En esta sección se tratará de explicar a fondo las tareas que resuelven cada una de ellas, para comprender su funcionamiento.

5.2.1. Faster-R-CNN

El objetivo de la red “Faster-R-CNN” en este modelo es el de obtener las regiones de interés de una imagen, ROI (Region of Interest). Para ello la tarea a resolver por la red Faster-R-CNN se divide en dos partes, primero obtener para cada objeto el cuadro delimitador correspondiente y en consecuencia, obtener su ROI. Una vez definido el cuadro delimitador, el objetivo es determinar, para cada ROI, la clase del objeto comprendido. Este objetivo se consigue ejecutando un proceso denominado “ROI pooling”, que consistiría en realizar la operación de *agrupamiento* (pooling) sobre dicha ROI.

Pero existe un problema, para realizar un *agrupamiento* sobre una ROI, el *desplazamiento* (stride) difícilmente coincidirá con el número de píxeles de dicha ROI.

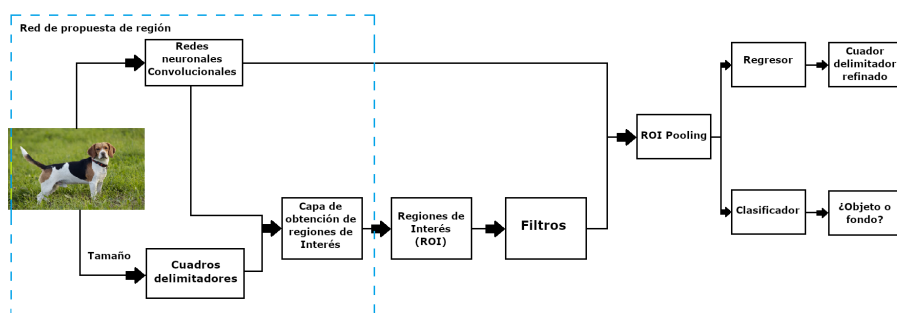


Figura 5.4: Pipeline de la red Faster-R-CNN.

Supongamos que a la hora de predecir un cuadro delimitador, obtenemos una celda de 17×17 píxeles y se quiere realizar un *agrupamiento* sobre dicha celda para obtener la clase del objeto. Se ha definido que la dimensión del filtro resultante después de la primera capa sea de 7×7 píxeles. Esto daría un *desplazamiento* de $17/7 = 2.42$ que redondeando toma el valor 2, lo cual no permitiría recorrer eficientemente la celda original (17×17 píxeles).

ROI Pooling: El desplazamiento es cuantificado

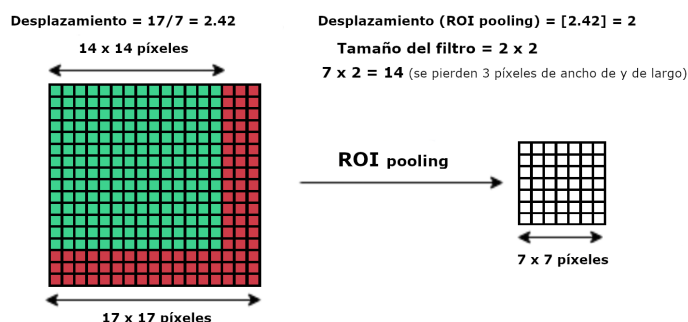


Figura 5.5: Representación visual de ROI Pooling 1/2.

Como se puede apreciar en la Figura 5.5, no se recorre eficientemente la celda, y se pierde información en el proceso.

Esta pérdida de información se soluciona aplicando una técnica innovadora denominada alineamiento de regiones de interés (ROI alignment). En este proceso, si el *desplazamiento* obtenido es un número racional, en vez de redondearlo al número entero más cercano como en el ejemplo anterior, se toma ese número sin modificar. De esta forma, se hacen filtros con este tamaño, y cada filtro se divide en cuatro partes, esquina superior izquierda, superior derecha, inferior izquierda e inferior derecha. Esto genera 4 subceldas, que son procesadas por un proceso de interpolación bilineal, el cual deja un valor numérico

para cada subcelda, y con esos 4 valores numéricos obtenidos en cada subcelda, se puede aplicar *agrupamiento de media* o *agrupamiento de máximo*.

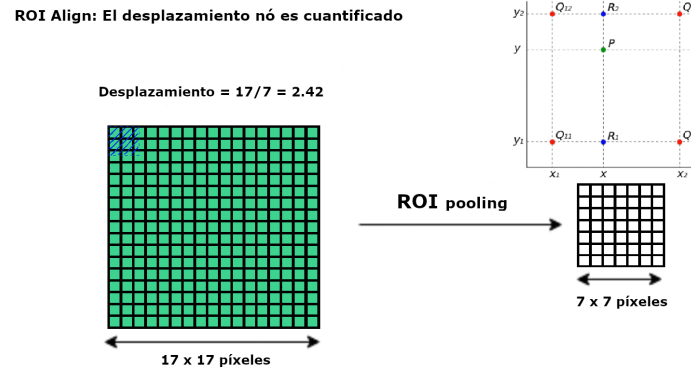


Figura 5.6: Representación visual de ROI Pooling 2/2.

En el caso de la celda de 17x17, si el *desplazamiento* es de 2.42, al hacer un filtro de 2.42 píxeles y dividirlo en 4 subceldas, esas subceldas contendrán algún píxel adicional, por ello, el proceso de interpolación bilineal lo que hace es asegurar un valor a esas subceldas a partir de esos píxeles, para que se pueda realizar el proceso de *agrupamiento* a partir de 4 valores numéricos.

5.2.2. Segmentación semántica usando FCN

Una vez obtenidas las diferentes ROI de la imagen y las clases de cada uno de los diferentes cuadros delimitadores, se utiliza una FCN (Fully connected network) para obtener la máscara. Estas redes son clave para este tipo de tareas ya que la labor de obtener la máscara requiere de un alineamiento de píxeles preciso. Las FCN son efectivas para este tipo de tarea ya que retienen la información espacial a nivel de píxel.

Siendo K , el número de clases predichas por nuestra red de Faster-R-CNN, para cada ROI, se obtienen K máscaras binarias (en blanco y negro), marcando en blanco la máscara de la actual clase i , siendo i desde $1...K$.

Cada ROI, por cada clase, se recorre fila por fila de izquierda a derecha, como si de un barrido de píxeles se tratara, y por cada píxel, se marca con un 1, si ese píxel pertenece a la máscara de la clase actual, o con un 0 si ese píxel pertenece a cualquier otra posible máscara o ningún objeto. Las figuras 5.7 y 5.8 muestran como una ROI genera diferentes máscaras para cada clase.



Figura 5.7: Visualización de obtención de una máscara sobre una ROI 1/2.



Figura 5.8: Visualización de obtención de una máscara sobre una ROI 2/2.

5.2.3. Función de error

Como se ha explicado anteriormente, se realizan tres tareas para cada ROI predicha, predecir el cuadro delimitador, la clase del objeto y predecir la máscara perteneciente del objeto. Como cada una de las tareas tiene una gran importancia, la formula de la función de error que se utiliza es la siguiente:

$$L = L_{clase} + L_{cuadroDelimitador} + L_{mascara} \quad (5.1)$$

A cada una de las tres predicciones se le da la misma relevancia. El error de clasificación L_{clase} y el error del cuadro delimitador $L_{cuadroDelimitador}$ son las mismas que se definieron en Fast-R-CNN [Ren et al., 2015]. Para el error de la máscara $L_{mascara}$, por cada ROI se obtiene como salida K máscaras binarias, donde K es el numero de clases que hay en dicha ROI. Se aplica la función “sigmoide” píxel por píxel y se define $L_{mascara}$ como la pérdida media de entropía cruzada binaria. Para un RoI asociado con el “ground-truth” para la clase k , $L_{mascara}$ solo se define en la máscara k -ésima, las otras máscaras de salida no contribuirían la función de error.

5.3. Arquitectura de la red

A nivel técnico la arquitectura de Mask-R-CNN está instanciada en dos arquitecturas diferentes:

- Por un lado está la denominada *columna vertebral* (backbone), la cual se encarga de extraer las características de la imagen a diferentes escalas.

- Después está la *cabeza*, encargada de la clasificación/regresión de los cuadros delimitadores y de la predicción de las máscaras.

5.3.1. Red piramidal de características

La arquitectura *columna vertebral* consiste en una *red piramidal de características* FPN [Lin et al., 2017] (Feature pyramid network). FPN utiliza una arquitectura de arriba hacia abajo con conexiones laterales para construir una pirámide de características en la red a partir de una entrada de escala única. La vía ascendente es la red convolucional habitual para la extracción de características. A medida que subimos, la resolución espacial disminuye. Con más estructuras de alto nivel detectadas, el valor semántico para cada capa aumenta. La figura 5.10 muestra como a cada nivel se va obteniendo más valor semántico pero se pierde resolución.

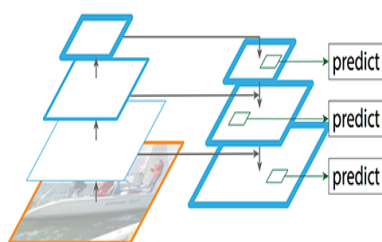


Figura 5.9: Representación en alto nivel de FPN. Figura obtenida de [Lin et al., 2017].

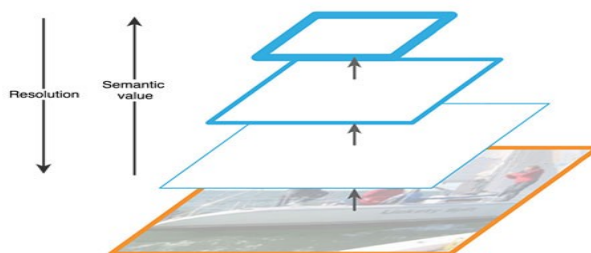


Figura 5.10: Esquema de como FPN a cada capa obtiene más información de valor semántico pero pierde más resolución. Figura obtenida de ¹.

La ruta ascendente consiste en una ResNet [Targ et al., 2016] (es un tipo de red neuronal la cual se explicará más adelante en la sección 5.3.2). Se compone de muchos módulos de convolución (conv_i para i desde 1 a 5) cada uno tiene muchas capas de convolución. A medida que avanzamos, la dimensión espacial se reduce en $1/2$, es decir, a cada módulo que se avanza el valor del *desplazamiento* se va duplicando. La salida de cada módulo de convolución se etiqueta como C_i y luego se utiliza en la ruta de arriba hacia abajo. La Figura 5.11 muestra detalladamente el proceso de ascensión y descenso y como estos se conectan mutuamente.

En el camino de arriba hacia abajo se aplica un filtro de convolución 1×1 a cada convolución C_i a 256-d (profundidad de 256 dimensiones) para crear el mapa correspondiente M_i . Estos mapas de características se utilizan para generar las predicciones. A medida que

¹https://medium.com/@jonathan_hui/understanding-feature-pyramid-networks-for-object-detection-fpn-45b227b9106c

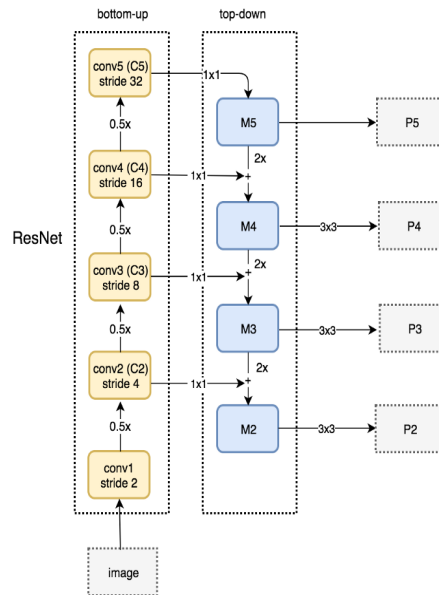


Figura 5.11: Representación visual de los procesos de ascender y descender en FPN. Figura obtenida de ².

se avanza por el camino de arriba hacia abajo, se aumenta la muestra de la capa anterior en 2 usando el muestreo de vecinos más cercano. Nuevamente se aplica una convolución 1×1 a los mapas de características correspondientes en la ruta ascendente. Se aplica una convolución 3×3 a todas las capas fusionadas para obtener las predicciones salvo en el mapa que proviene de la convolución 5, M5.

5.3.2. ResNet 101

La arquitectura ResNet [Targ et al., 2016], al igual que la “AlexNet” [Alom et al., 2018] supuso un gran revolución en el campo de *visión por computador*. Antes de que surgiera la arquitectura “ResNet”, uno de los problemas que existía en las arquitecturas de entonces, es que cuanto más capas se intentara añadir a una red, el error que se iba propagando iba reduciéndose hasta volverse ineficiente, lo que provocaba que a mayor número de capas el error de la red convergía.

La idea característica de ResNet fue crear conexiones entre capas que preservasen este valor de error a lo largo de la red, haciendo que este valor no se reduzca significativamente. Desde que surgió este concepto, han ido surgiendo varias variantes de ResNet con diferentes número de capas. Existe la ResNet 50 la cual contiene 50 capas y la ResNet 101 la cual contiene 101 capas. El modelo Mask-R-CNN hace uso de la ResNet 101 así

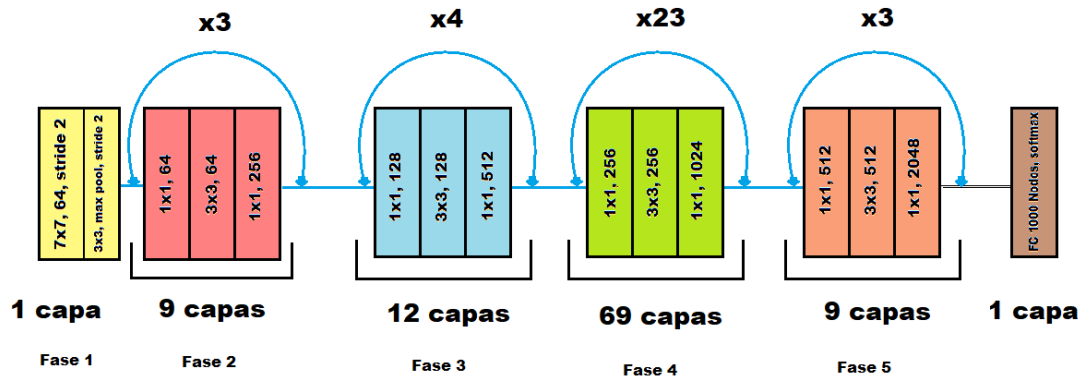


Figura 5.12: Representación visual de la arquitectura ResNet 101.

que procederemos a detallar ésta.

Como se puede apreciar en la Figura 5.12, la arquitectura está dividida en 5 fases.

1. La primera fase consiste en una capa de convolución utilizando un filtro de 7×7 y un desplazamiento de 2, seguida de un *agrupamiento de máximo* de 3×3 con un desplazamiento de 2.
2. La segunda fase consiste en una capa de 1×1 y 64-d(dimensiones) seguida de una capa 3×3 y 64-d y seguida de una capa 1×1 y 256-d, estas capas se repiten 3 veces, obteniendo **9 capas** en total y entre cada conjunto de estas hay una conexión intermedia.
3. La fase 3 consiste en una capa de 1×1 y 128-d seguida de una capa 3×3 y 128-d y seguida de una capa 1×1 y 512-d, estas capas se repiten 4 veces, obteniendo **12 capas** en total y entre cada conjunto de estas hay una conexión intermedia.
4. La fase 4 consiste en una capa de 1×1 y 256-d seguida de una capa 3×3 y 256-d y seguida de una capa 1×1 y 1024-d, estas capas se repiten 23 veces, obteniendo **69 capas** en total y entre cada conjunto de estas hay una conexión intermedia.
5. Por último, la fase 5 consiste en una capa de 1×1 y 512-d seguida de una capa 3×3 y 512-d y seguida de una capa 1×1 y 2048-d, estas capas se repiten 9 veces, obteniendo **9 capas** en total y entre cada conjunto de estas hay una conexión intermedia. Finalmente en la ultima capa se realiza un *agrupamiento de media* con una

capa completamente conectada la cual contiene 1000 nodos y en ella se realiza la función *softmax*.

5.4. Estado del arte

El modelo Mask-R-CNN [He et al., 2017] se ha entrenado sobre la base de datos COCO [Veit et al., 2016], la cual se explica más detalladamente en el Anexo A.1. Para el proceso de entrenamiento se utilizaron 80.000 imágenes como conjunto de entrenamiento, 35.000 imágenes como conjunto mixto de entrenamiento/validación y 5.000 como conjunto de validación únicamente.

Para evaluar la calidad de los resultados de “Mask-R-CNN” la documentación presenta una comparativa con respecto al modelo FCIS [Li et al., 2017] (Segmentación semántica completamente convolucional consciente de la instancia). Este último modelo, ganó los desafíos de segmentación organizados usando la de base de datos COCO en 2015 y en 2016, lo cual implicaría realizar una comparación con la mejor tecnología contemporánea en la predicción de poses.

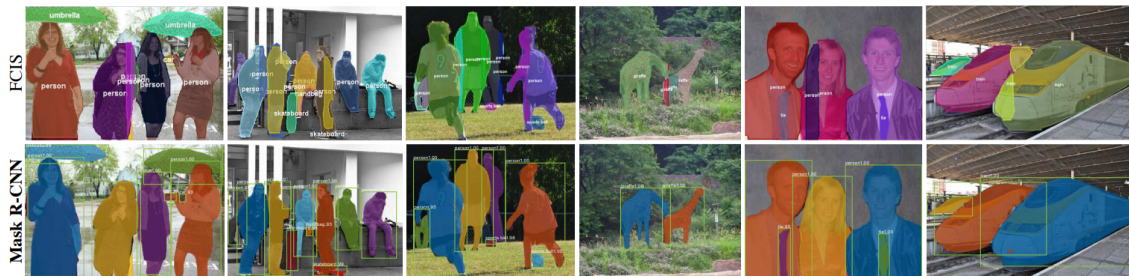


Figura 5.13: Comparativa entre las predicciones realizadas por FCIS (arriba) y Mask-R-CNN (abajo), Figura obtenida de [He et al., 2017].

Como se puede ver en la Figura 5.13, los resultados son considerablemente más precisos.

6. CAPÍTULO

Extensión del modelo a la detección de pose de pelotaris

6.1. Detección de poses en Pelota vasca y Cesta punta

Nuestro objetivo era evaluar y si fuera necesario mejorar el desempeño del modelo seleccionado para la detección de pose de pelotaris en Pelota vasca y Cesta punta.

La Pelota Vasca y la Cesta punta son deportes tradicionales vascos que se caracterizan por ser un estilo de juego completamente diferente a los deportes más comunes que se practican en el del mundo. Ambos deportes se practican en un mismo entorno denominado frontón el cual se caracteriza por dos paredes, una frontal y otra situada a la izquierda, ambas de color verde. La diferencia entre ambos deportes es que en Cesta punta se utiliza una herramienta llamada cesta que se utiliza para lanzar la bola mientras que en Pelota vasca se utiliza la propia mano sin ninguna herramienta, portan una ligera protección que cubre la mano pero es visualmente irrelevante para este análisis ya que hay veces que no se distingue. Las poses que toman los jugadores (pelotaris) se alejan de las poses comunes en la vida diaria pues tratan de golpear/lanzar las pelotas de una forma que no se realiza en ninguna otra parte del mundo, dichas poses son únicas y difícilmente registradas en bases de datos como la de COCO.

Por esta razón era de interés investigar el uso de los modelos de detección de pose en este problema particular. Además, existe un interés adicional al tratarse de un deporte local que muy probablemente no haya sido analizado por ningún otro modelo.

Los pasos que se realizaron para el análisis y mejora del modelo fueron los siguientes:

1. Recopilación de imágenes de pelotaris jugando y posando en un frontón.
2. Ejecución del modelo extraído en estas imágenes.
3. Detección y caracterización de los diferentes fallos o imperfecciones.
4. Anotación manual del etiquetado del sub-conjunto de imágenes obtenidas.
5. Re-entrenamiento sobre el subconjunto etiquetado manualmente.
6. Análisis de la mejora realizada tras el proceso de re-entrenamiento.

6.2. Recopilación de imágenes y evaluación del modelo sobre las mismas

Antes que tratar detectar un problema o un comportamiento imperfecto, había que obtener las imágenes sobre las que trabajar. Se descendieron de forma manual 50 imágenes de Pelota vasca y 50 imágenes de Cesta punta, todas desde el buscador de “google imágenes”. La idea era buscar imágenes con diferentes resoluciones, con variedad de posturas complejas y con personas superpuestas unas sobre otras. Generalmente las imágenes contenían uno o dos pelotaris, pero en algunos casos había hasta 4 pelotaris. En el conjunto de 100 imágenes el número de pelotaris en cada imagen fue el siguiente:

	Pelota vasca	Cesta punta
1 Persona	11	17
2 Personas	14	15
3 Personas	10	10
4 Personas	8	5
Otros ¹	7	3

Tabla 6.1: Cantidad de personas por imagen.

Después se ejecutó el modelo sobre estas imágenes. El modelo se puede obtener desde el siguiente repositorio de Github: ². Para lograr ejecutar el modelo sobre las imágenes hubo que modificar el código del script que ejecuta el modelo, pues este tan solo ejecutaba

¹En “otros” se encuentran aquellas imágenes especiales con personas superpuestas, cuerpos que no aparecen completos e imágenes con más de 4 personas

²<https://github.com/chrispolo/Keypoints-of-humanpose-with-Mask-R-CNN>

una imagen aleatoria de un directorio tras cargar los pesos de la red y hacer las demás configuraciones necesarias. Se modificó el código para que recorriera todas las imágenes de un directorio dado y que para cada imagen realizara la predicción. Se dividió el conjunto de imágenes de Pelota vasca y Cesta punta en dos directorios diferentes para así poder ejecutar la red en diferentes scripts y analizar los resultados obtenidos de forma organizada.

6.3. Análisis de los resultados del modelo

Para detectar algún posible comportamiento imperfecto o poco refinado sobre el cual poder realizar una posible mejora, primero; se ejecutó el modelo sobre el directorio de imágenes de Pelota vasca y después sobre el directorio de Cesta punta, para así analizarlas separadamente.

Se observó que en términos generales el modelo predijo las poses acertadamente tanto en las imágenes de Pelota vasca como las de Cesta punta. Véanse las figuras 6.1 y 6.2 las cuales ilustran dos ejemplos de predicción acertada.



Figura 6.1: Ejemplo de predicción correcta sobre Pelota vasca.

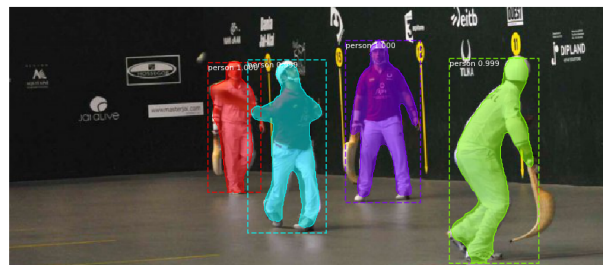


Figura 6.2: Ejemplo de predicción correcta sobre Cesta punta.

6.3.1. Errores en Pelota vasca

Pero aun así, ciertas imágenes de Pelota vasca contenían un patrón erróneo en común. Entre ellas identificamos y caracterizamos tres tipos de errores diferentes:

- I) Extensión imperfecta de máscaras al haber una superposición de personas entre sí, véanse los ejemplos de las figuras 6.3 y 6.4.

- II) Exclusión de brazos o piernas al realizar una predicción de máscara más reducida de lo que debería, véanse los ejemplos de las figuras 6.5 y 6.6.
- III) Interpretar erróneamente que una de las manos pertenece a otra persona y clasificar dicha mano como si de otra persona se tratase, véase como el ejemplo de la Figura 6.7 ilustra este error.



Figura 6.3: Ejemplo del refinamiento impreciso en Pelota Vasca error I.



Figura 6.4: Ejemplo del refinamiento impreciso en Pelota Vasca error I.



Figura 6.5: Ejemplo del refinamiento impreciso en Pelota Vasca error II.

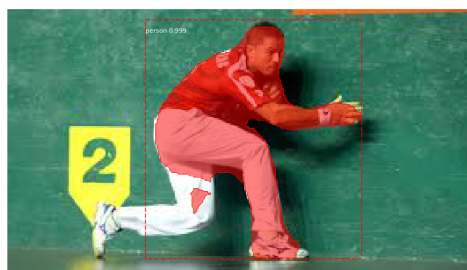


Figura 6.6: Ejemplo del refinamiento impreciso en Pelota Vasca error II.

Tras la visualización de estos errores, se ha decidido denominar este tipo de error como “Error de refinamiento impreciso”.

6.3.2. Errores en Cesta punta

Por otro lado, al procesar las imágenes de Cesta punta fue más frecuente detectar errores de refinamiento impreciso. Sin embargo, tan solo se halló un tipo de error. Este error es más grave que los detectados en Pelota vasca pues destaca más a la vista, se trata de una



Figura 6.7: Ejemplo del refinamiento impreciso en Pelota Vasca error III.

extensión de la máscara predicha en la que anexa la cesta que portan los jugadores con el cuerpo, véanse las figuras 6.8 y 6.9 las cuales ilustran este problema. Luego en una imagen el problema se aprecia de manera más acentuada al clasificar erróneamente la cesta como si de una persona adicional se tratase. La imagen en cuestión se muestra en la Figura 6.10. Estos errores muestran la necesidad de adaptar el aprendizaje de las redes para el caso de imágenes de este deporte.

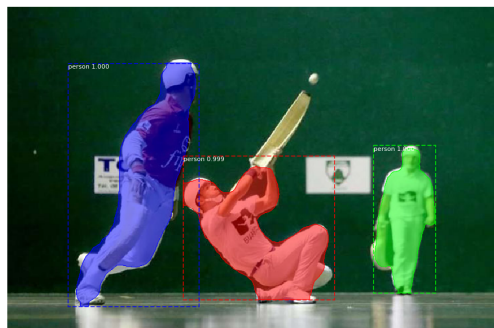


Figura 6.8: Ejemplo del refinamiento impreciso en Cesta punta 1/2.



Figura 6.9: Ejemplo del refinamiento impreciso en Cesta punta 2/2.

6.4. Mejoras realizadas al modelo

Una vez detectadas las deficiencias del modelo, se contemplaron varias ideas con respecto al tipo de mejora a introducir:

- Re-entrenar el modelo con un subconjunto menor de imágenes de Cesta punta y Pelota vasca, para ello habría que ser capaz de entrenar el modelo y generar un conjunto de etiquetas, en el formato utilizado por COCO, para dichas imágenes.

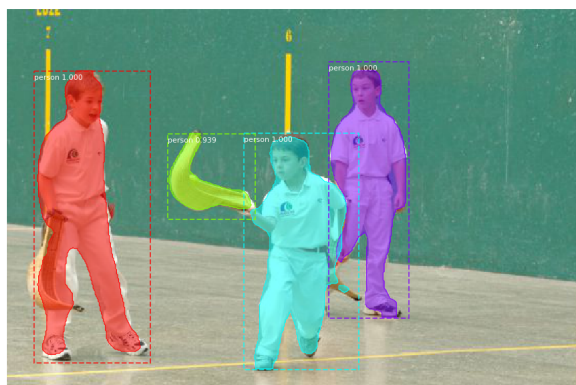


Figura 6.10: Ejemplo de la clasificación imprecisa en Cesta punta.

- Obtener la imagen de salida como una entrada para una red neuronal menor de tipo Perceptrón multicapa, para analizar ya sea el cuadro delimitador o la clase predicha y descartarla en caso de error.
- Re-entrenar el modelo, pero modificando la formula de la función de error, para darle mayor relevancia a la detección correcta de objetos relacionados con los dos deportes, por ejemplo, la Cesta punta.

Antes de tratar de realizar alguna mejora, se decidió tratar de ejecutar el proceso de entrenamiento utilizando la base de datos original con la que fue entrenada. Fue imposible realizar el entrenamiento con el ordenador disponible, posiblemente por el volumen de la cantidad de imágenes ya que en total eran 27Gb. O bien el problema pudo deberse al alto número de parámetros de la red.

Debido a este percance, se trató de ejecutar el proceso de entrenamiento en Google Colab, pero esto tampoco fue posible, pues no se logró subir dicha cantidad de imágenes al sitio, ni tampoco se logró subir esa cantidad de imágenes a un repositorio como GitHub sobre el cual poder extraer las imágenes.

Como solución alternativa se decidió tratar de re-entrenar el modelo con el subconjunto menor de imágenes mencionado anteriormente, porque si el problema se debía al volumen de datos, de este modo, podría ser posible re-entrenar el modelo. Para ello, fue necesario anotar las imágenes en el formato COCO, es decir, siguiendo la sintaxis desarrollada en el Anexo [A.2](#).

6.4.1. Anotación manual del etiquetado del sub-conjunto de imágenes obtenidas

Una de las tareas necesarias para llevar a término este proyecto ha sido el etiquetado de imágenes de Pelota vasca y Cesta punta. Este conjunto de imágenes podría usarse en el futuro para mejora aún más el desempeño de los modelos. Existen varios métodos para anotar las imágenes, pero se identificó una herramienta ofrecida por los desarrolladores de COCO denominada “COCO-annotator”, la cual debe ser instalada y se ejecuta desde el navegador. Gracias a esta herramienta no solo fue posible anotar manualmente las imágenes de forma muy precisa, sino que además, fue posible exportar las anotaciones en un fichero “.json” para cada base de datos. Asimismo la herramienta también nos permitió importar anotaciones en el mismo formato para poder anexar bases de datos. Las Figuras 6.5 y 6.6 muestran la interfaz gráfica de la herramienta “COCO-annotator”.

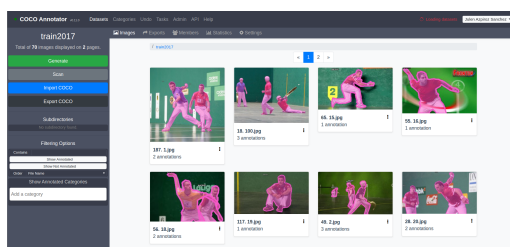


Figura 6.11: Visualización de 'COCO-annotator' 1/2.

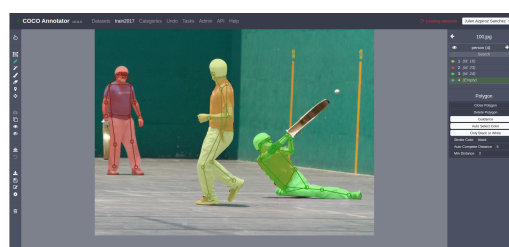


Figura 6.12: Visualización de 'COCO-annotator' 2/2.

Para el proceso de etiquetado de las imágenes, se dividió el conjunto de 100 imágenes de las que se disponía en un 70% para el conjunto de entrenamiento, 10% para el conjunto de validación y el 20% restante para el conjunto de prueba. Por cada humano que apareciese en cada imagen, se anotaban dónde se encontraban las articulaciones del cuerpo, y se dibujaba manualmente la máscara de los mismos. Aproximadamente se tardaba entre 2 y 3 minutos en anotar un humano por completo. En total se anotaron 250 humanos aproximadamente, como se ha mencionado anteriormente, había imágenes que solo contenían una persona, pero en otras, aparecían hasta seis personas.

6.4.2. Re-entrenamiento del modelo utilizando el subconjunto de imágenes anotado manualmente

Tras haber terminado de anotar las imágenes, se procedió a realizar el proceso de re-entrenamiento. El script de entrenamiento en un punto del código obtiene el fichero de

pesos que se va a usar, si se le introduce un fichero no existente, genera uno nuevo, es decir, realizará un proceso de entrenamiento desde el principio. Si se le introduce un fichero ya existente de pesos, entonces realizará el proceso de entrenamiento partiendo del estado de ese fichero, por lo que se ejecutará un re-entrenamiento. Utilizando esta información, únicamente fue necesario utilizar como entradas del entrenamiento el fichero de pesos y la base de datos recién-anotada junto a los ficheros de anotaciones.

Pese a utilizar un conjunto nuevo con un número menor de imágenes el proceso de entrenamiento tampoco fue posible realizar desde el ordenador del que se disponía, una vez descartado el problema de la cantidad de imágenes, seguramente el problema sea debido al gran número de parámetros en la red. Por ello, se decidió realizar el proceso de re-entrenamiento en el entorno de desarrollo de GoogleColab.

El proceso de entrenamiento está dividido en tres fases, y cada fase se entrena en un número determinado de épocas. La fase 1 es para entrenar la red *cabeza* mencionada en la Sección 5.3 que se encargaba de la clasificación/regresión de los cuadros delimitadores y de la predicción de las máscaras. Las fases 2 y 3 para entrenar la ResNet que se encargaba de obtener las características de las imágenes. En la fase 2 del entrenamiento se entrena la ResNet a partir de su 4 fase (a partir de las 69 capas que se menciona en la Sección 5.3.2) y en la fase 3 del entrenamiento se entrena a partir de la fase 3 de la red ResNet, la Figura 5.12 ilustra la representación visual de la arquitectura ResNet 101.

Cada época tardó alrededor de 40 minutos en completarse y el fichero original estaba configurado para que se entrenasen 15 épocas en la primera fase, 5 en la segunda y 80 en la tercera. Fueron ejecutados 3 entrenamientos diferentes partiendo desde el número mínimo de épocas, que serían una para cada fase hasta alcanzar el valor “óptimo” que consistió en 3 épocas para la primera fase, 2 para la segunda y 5 para la tercera. El problema de realizar el entrenamiento desde GoogleColab es que al ser un entorno de desarrollo por conexión remota, procesos largos como el de ejecutar un entrenamiento, se deben finalizar sin que se pierda la conexión con el servidor, ya que si se pierde la conexión el proceso se interrumpe y se tendría que repetir desde cero. Teóricamente el entorno de desarrollo permite al usuario estar hasta 12 horas conectado, pero más de una vez la conexión se perdió al cabo de llevar 3 horas de conexión. Se podía haber intentado realizar 15 épocas en total (20 no serían posible por que excederían las 12 horas) o algo similar, pero se consideró correcto haber alcanzado 10 épocas de entrenamiento.

6.5. Análisis de los resultados

Tras haber alcanzado este punto, ya se contaba con un fichero de pesos relativamente modificado utilizando el cual se podía realizar una comparativa. Se realizaron dos tipos diferentes de evaluación, primero se compararon de forma cuantitativa los resultados obtenidos utilizando ambos ficheros de pesos que serían: el entrenado previamente con las imágenes de COCO, y el fichero re-entrenado con las imágenes anotadas de Pelota vasca y Cesta punta. Después se analizó de forma visual, si dichos cambios son perceptibles o no y si se destaca alguna mejora.

6.5.1. Evaluación cuantitativa

El sistema de evaluación se implementa siguiendo unas métricas definidas por los desarrolladores de COCO. Se procederá a explicar brevemente cómo se expresan las métricas para poder mostrar los datos y que sean comprensibles.

Existen 3 conceptos dentro de esta métrica:

1. **Precisión:** Indica cuánto de precisa ha sido una predicción sobre todos los casos positivos. Media de Precisión lo denominaremos como **AP** (average precision).
2. **Recuerdo:** Indica cómo de bien se ha predicho sobre todos los casos reales. Media de recuerdo lo denominaremos como **AR** (average recall).
3. **Intersección sobre Unión(IoU)** Porcentaje para medir la superposición entre las dos máscaras (la predicha y la real).

La *precisión* se obtiene como un porcentaje que indica como de acertado es un tipo de predicción sobre todos los predichos positivos. Supongamos un ejemplo de tratar de predecir X personas sobre 100 si tienen cáncer, el porcentaje de *precisión* sería cuántos diagnosticados como enfermos con cáncer han sido predichos correctamente, diferenciando entre aquellos con cáncer predichos correctamente y aquellos sin cáncer predichos como que sí tienen cáncer. La fórmula del porcentaje de *precisión* sería la siguiente:

$$Precisin = \frac{VerdaderosPositivos(VP)}{TodosLosPacientesPositivos(VP + FP)} \quad (6.1)$$

Por otro lado el *recuerdo* se obtiene como otro porcentaje que indica cómo de correcto han sido los positivos predichos, en el ejemplo anterior del cáncer, sería cuántas personas con cáncer han sido diagnosticadas correctamente, diferenciando entre aquellas con cáncer predichas acertadamente y aquellas con cáncer no predichas. La fórmula del porcentaje de *recuerdo* sería el siguiente:

$$\text{Recuerdo} = \frac{\text{VerdaderosPositivos}}{\text{VerdaderosPositivos} + \text{Falsosnegativos}} \quad (6.2)$$

El **IoU** se obtiene como resultado de aplicar la división del área superpuesta, la cual sería la unión entre el área predicha y el área real, y el área real, tal y como se muestra en la Figura 6.13.

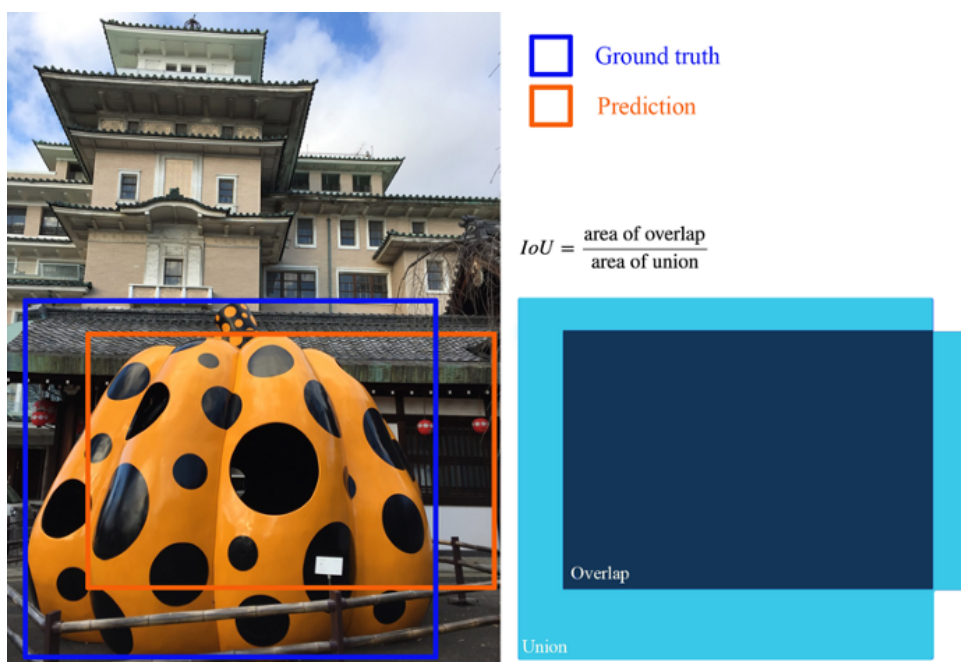


Figura 6.13: Representación visual de obtención de IoU. Figura obtenida de ³

Las estadísticas de ejecutar la evaluación en la red con los diferentes pesos se muestran en las tablas 6.2 y 6.3. La Tabla 6.2 muestra las estadísticas del fichero de pesos antes de ser re-entrenado, y la Tabla 6.3 muestra las estadísticas del fichero re-entrenado.

Estas estadísticas se obtienen tras ejecutar una función del modelo, a la cual se le pasa el directorio de imágenes con sus anotaciones y un fichero de pesos. La función evalúa ese fichero de pesos sobre la base de datos asignada. La base de datos sobre la que se evalúa

³https://medium.com/@jonathan_hui/map-mean-average-precision-for-object-detection-45c121a31173

Predicción	IoU	Área	Porcentaje
AP	50:0.95	Todas	44.6 %
AP	50	Todas	76.7 %
AP	75	Todas	53.9 %
AP	50:0.95	Pequeñas	20.2 %
AP	50:0.95	Medianas	54.0 %
AP	50:0.95	Grandes	38.7 %
AR	50:0.95	Todas	50.2 %
AR	50:0.95	Pequeñas	20.0 %
AR	50:0.95	Medianas	56.4 %
AR	50:0.95	Grandes	45.8 %

Tabla 6.2: Estadísticas del modelo antes del re-entrenamiento.

Predicción	IoU	Área	Porcentaje
AP	50:0.95	Todas	48.3 % + 3.7 %
AP	50	Todas	80.5 % + 3.8 %
AP	75	Todas	62.3 % + 8.4 %
AP	50:0.95	Pequeñas	10.1 % - 10.0 %
AP	50:0.95	Medianas	59.8 % + 5.8 %
AP	50:0.95	Grandes	41.4 % + 2.7 %
AR	50:0.95	Todas	52.0 % + 1.8 %
AR	50:0.95	Pequeñas	10.0 % - 10.0 %
AR	50:0.95	Medianas	60.7 % + 4.3 %
AR	50:0.95	Grandes	45.8 % + 0 %

Tabla 6.3: Estadísticas del modelo tras ejecutar el re-entrenamiento.

es el conjunto de prueba anotado manualmente el cual contenía 20 imágenes (10 de Pelota vasca y 10 de Cesta punta).

Como se puede apreciar en las tablas 6.2 y 6.3 primero se realizan 6 cálculos que obtienen valores diferentes para **AP** en diferentes contextos y después 4 cálculos para **AR**. Cuando en la segunda columna aparece “50:0.95”, quiere decir que recoge una media de la precisión (**AP**) o del recuerdo (**AR**) obtenido en un conjunto de iteraciones, donde se empieza desde un **IoU** de 0.05 hasta alcanzar 0.95, avanzando 0.05 en cada iteración. En la gran mayoría de cálculos, se obtiene la media realizando este mismo proceso salvo en

dos cálculos de **AP** que recoge los valores obtenidos en aquellas predicciones que estén por encima de un IoU superior a 0.50 o 0.75 (línea 2 y 3 respectivamente).

En la tercera columna muestra para qué tipos de área realiza ese cálculo, generalmente hace el cálculo en todas las áreas, pero como se puede apreciar, en algunos casos se realiza para áreas pequeñas, áreas medianas o áreas grandes.

En general el modelo re-entrenado mejora un poco en cada caso, salvo que al parecer en las áreas pequeñas los porcentajes son un 10 % menos acertados, tanto en *predicción* como en *recuerdo*. Pero en general el modelo mejora las predicciones alcanzando un **8.4 %** para aquellas áreas con **IoU** superior a 0.75.

6.5.2. Análisis visual

Se ejecutaron de forma separada los modelos con los conjuntos de pesos sobre el directorio de imágenes de prueba, para tratar de evaluar los resultados obtenidos a nivel visual, ya que es posible que pese a que las métricas obtenidos sean mejores, el modelo realice algún comportamiento extraño o imprevisto.

Del total de 20 imágenes se aprecia que las máscaras obtenidas con el conjunto de pesos inicial sobresalían ligeramente en algunas personas, mientras que con el conjunto de pesos re-entrenado esas mismas máscaras sobresalían ligeramente menos. Sin embargo, hay imágenes en las que se destaca una mejora palpable. véase la diferencia entre la Figura 6.7 y la 6.8, donde se puede apreciar como la persona que está más a la izquierda en la Figura 6.7 se le anexa la cesta al cuerpo y en la Figura 6.8 esto no ocurre.



Figura 6.14: Ejemplo de mejora del modelo 1/2.



Figura 6.15: Ejemplo de mejora del modelo 2/2.

Teniendo en cuenta los resultados obtenidos tras realizar la evaluación del modelo, y viendo las diferencias a nivel visual entre los dos modelos, se percibe una mejora. Es cierto que no es una mejora significativa, pero teniendo en cuenta que hemos realizado el

entrenamiento sobre 70 imágenes, y que además la comparativa solo se ha realizado sobre 20 imágenes se trata de una mejora tangible. Asimismo, estos resultados nos confirma que la red que se entrenó en un principio está abierta a mejoras y no está sujeta a un dominio de imágenes preciso.

7. CAPÍTULO

Conclusiones

Dos de las tareas más importantes del proyecto han sido el estudio de los modelos más influyentes para la detección de la pose de un objeto, y la implementación de la mejora sobre el modelo adquirido.

Desde la perspectiva del estudio, hemos comprobado que los modelos para la detección de pose han ido progresando eficientemente, no solo en los resultados, sino también en la complejidad técnica de su funcionamiento. Esto demuestra que el coste de generar nuevos modelos es cada vez más alto y al mismo tiempo resultará más difícil innovar en este campo tratando de desarrollar un modelo nuevo debido a las limitaciones de equipamiento.

Por otro lado, tomando una perspectiva más interna, como es la de trabajar desde un modelo, hemos demostrado que pese a la gran magnitud de esta red y su base de datos, ésta se puede refinar extendiendo el ámbito de imágenes sobre las que entrenar y que además resulta posible reconocer poses de pelotaris en Pelota vasca/Cesta punta.

El proyecto nos ha permitido identificar diferentes tipos de errores de los modelos neuronales en detección de poses en Pelota vasca y Cesta punta. Este tipo de análisis no ha sido realizado con anterioridad o al menos no encontramos referencias a este hecho en la literatura.

Posiblemente se habrían podido realizar más mejoras si se hubiera contado con un entorno de trabajo más avanzado, a la altura del que disponían los desarrolladores del mismo. Éste ha sido el obstáculo fundamental para avanzar más en la consecución de los objetivos iniciales del proyecto.

Finalmente, hemos descubierto desde diferentes perspectivas, que se pueden generar modelos nuevos aplicando técnicas nuevas o bien innovando en otros detalles menores como puede ser el modo de representar la salida; y que además, por muy complejo y eficiente que pueda resultar un modelo, como es el caso de “Mask-R-CNN”, se puede afirmar con certeza que dicho modelo se podrá mejorar si se tienen en cuenta las características particulares del dominio de aplicación.

Anexos

A. ANEXO

Base de datos COCO

A.1. Descripción

El modelo “Mask-R-CNN” [He et al., 2017] se ha entrenado sobre la base de datos de COCO [Veit et al., 2016], una base de datos enfocada a la detección de objetos, detección de pose y la segmentación de instancias. Contiene 330.000 imágenes de las cuales 200.000 están etiquetadas. Es una base de datos accesible desde su Web cocodataset.org, la cual lleva vigente desde 2014 y a cada año que transcurre se va actualizando.

Además de lo mencionado anteriormente, tiene 80 categorías de objetos clasificados con un total de 1.5 millones instancias entre todas las categorías.

A.2. Etiquetas

Cada base de datos, utiliza diferentes formas de representar las anotaciones. Es importante saber como están realizadas las anotaciones, no solo para entender como está clasificada cada imagen, sino también para poder realizar una modificación o un aditivo a la base de datos para poder re-entrenar de una manera diferente, implementar una modificación, etc.

Las anotaciones están en ficheros “.json”, los cuales contienen un peso de memoria relativamente ligero, pero están en un formato demasiado comprimido como para poder revisarlos. Los fichero ocupan entre 3.9 Mb y 490 Mb, generalmente las de mayor peso son las anotaciones de los datos de entrenamiento, los datos de validación y de test son los

que menos espacio ocupan. El problema es que si se trata de abrir con un editor de texto corriente, el fichero más ligero (3.9Mb), el ordenador se bloquea y es incapaz de abrirlo, y siempre se ha de forzar el cierre del editor de texto. Por sus dimensiones y el formato utilizado para su creación estos archivos no pueden ser visualizados con los editores de texto tradicionales.

Utilizando otros medios se ha podido determinar que las etiquetas están clasificadas en 5 categorías. Cada categoría puede ser instanciada varias veces. Por ejemplo, en el caso de la categoría 2, que es la información con respecto a las licencias, por cada licencia diferente que hubiese en la base de datos, habría esa misma cantidad de instancias en la categoría de licencias, lo mismo ocurriría con imágenes, etc.

1. **Info:** Esta categoría contiene información general de la base de datos.

Variable	Descripción de la variable	Ejemplo
decription:	El nombre de la base de datos	“COCO 2017 DataSet”
url:	El url que da acceso la base de datos.	“http://cocodataset.org”
version:	Versión de la base de datos.	“1.0”
year:	Año de creación de la base de datos.	2017
contributor:	Representa el grupo de COCO que ha generado la base de datos.	“COCO-Consortium”
date_created:	Fecha en la que fue creada la base de datos.	“2017/09/01”

Tabla A.1: Información general de la base de datos.

2. **Licenses:** Esta categoría describe las licencias usadas en la base de datos, hay hasta 8 tipos de licencias diferentes.

Variable	Descripción de la variable	Ejemplo
url:	El url que da acceso a la licencia correspondiente.	“http://creative... ¹ ”
id:	El número que identifica la licencia.	4
name:	El nombre de la licencia.	“Attribution License”

Tabla A.2: Información de las licencias.

¹<http://creativecommons.org/licenses/by/2.0/>

3. **Images:** Esta categoría contiene información acerca de la imagen original, toda la información relevante antes de ser procesada.

Variable	Descripción de la variable	Ejemplo
id:	Identificador de la imagen. Sirve para relacionar imágenes con anotaciones.	242287
license:	El número de id de la licencia que utiliza esta imagen.	4
coco_url:	El url que da acceso a la imagen desde el dominio de la web de COCO.	“ http://images.c...² ”
flickr_url:	Similar a “coco_url”, solo que da acceso a la misma imagen, desde staticflickr.com.	“ http://farm3.stat...³ ”
width:	Ancho en píxeles de la imagen.	426
height:	Altura en píxeles de la imagen.	640
file_name:	Nombre del fichero.	“000000242287.jpg”
data_captured:	Fecha exacta en la que la imagen fue obtenida.	“2013-11-15.02:41:42”

Tabla A.3: Información de las imágenes.

4. **Annotations:** Aquí es donde se guarda la información correspondiente a las etiquetas. Una imagen, puede tener más de una agrupación de etiquetas, pues cada una representaría un objeto anotado en la imagen. Todos los valores de esta categoría son *enteros*.
5. **Categories:** Este campo almacena la información relativa a las categorías de los objetos. Como se ha mencionado anteriormente, COCO contiene hasta 80 categorías registradas, y algunas de ellas están agrupadas en una jerarquía de “super categorías”, para ordenarlas de forma eficiente. Por ejemplo, una super categoría sería animales, y dentro de esta, estarían diferentes categorías menores como perro, gato, caballo, etc.

El modelo sobre el que se trabaja en este proyecto no hace uso de esta categoría porque tan solo actúa sobre humanos y resulta irrelevante almacenar esa información para el mismo.

Las Figuras A.1 y A.2 ilustran un ejemplo mas visual de un fichero de anotaciones siguiendo la sintaxis mencionada:

²<http://images.cocodataset.org/val2017/000000242287.jpg>

³http://farm3.staticflickr.com/2626/4072194513_edb6acf2b_z.jpg

Variable	Descripción de la variable	Ejemplo
id:	El id de la anotación correspondiente.	125686
category_id:	Categoría a la que pertenece el objeto clasificado	2
iscrowd:	Variable binaria que se utiliza para identificar si hay una multitud la cual sería irrelevante de anotar manualmente.	0
segmentation:	Cadena de <i>enteros</i> que si se sigue en el orden que aparecen se genera la máscara del objeto.	[[164.81, 417.51, 164.81, ..., 134.59]]
image_id:	El id de la imagen en la que se hace la anotación.	242287
area:	El valor numérico del área de la máscara en unidad de píxeles.	42061.803400000001
bbox:	Cuatro valores numéricos que representan el cuadro delimitador del objeto.	[19.23, 383.18, 314.5, 244.46]
data_captured:	Fecha exacta en la que la imagen fue obtenida.	"2013-11-15.02:41:42"

Tabla A.4: Información de las anotaciones de cada objeto.

Variable	Descripción de la variable	Ejemplo
supercategory:	El nombre de la categoría superior.	"animal"
id:	El id que identifica la categoría.	22
name:	El nombre de dicha categoría.	"elephant"

Tabla A.5: Información de las categorías.

```
{
  "info": {
    "description": "COCO 2017 Dataset",
    "url": "http://cocodataset.org",
    "version": "1.0",
    "year": 2017,
    "contributor": "COCO Consortium",
    "date_created": "2017/09/01"
  },
  "licenses": [
    {
      "url": "http://creativecommons.org/licenses/by/2.0/",
      "id": 4,
      "name": "Attribution License"
    }
  ],
  "images": [
    {
      "id": 242287,
      "license": 4,
      "coco_url": "http://images.cocodataset.org/val2017/000000242287.jpg",
      "flickr_url": "http://farm3.staticflickr.com/2626/4872194513_edb6acfb2b_z.jpg",
      "width": 426,
      "height": 640,
      "file_name": "000000242287.jpg",
      "date_captured": "2013-11-15 02:41:42"
    },
    {
      "id": 245915,
      "license": 4,
      "coco_url": "http://images.cocodataset.org/val2017/000000245915.jpg",
      "flickr_url": "http://farm1.staticflickr.com/88/211747310_f58a16631e_z.jpg",
      "width": 500,
      "height": 333,
      "file_name": "000000245915.jpg",
      "date_captured": "2013-11-18 02:53:27"
    }
  ]
}
```

Figura A.1: Ejemplo visual de las etiquetas de como se representarían de forma ordenada.



Figura A.2: Ejemplo visual de una imagen con la variable `iscrowd` igual a 1.

Bibliografía

- [Alom et al., 2018] Alom, M. Z., Taha, T. M., Yakopcic, C., Westberg, S., Sidike, P., Nasrin, M. S., Van Esesn, B. C., Awwal, A. A. S., and Asari, V. K. (2018). The history began from alexnet: A comprehensive survey on deep learning approaches. *arXiv preprint arXiv:1803.01164*.
- [Carreira et al., 2016] Carreira, J., Agrawal, P., Fragkiadaki, K., and Malik, J. (2016). Human pose estimation with iterative error feedback. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4733–4742.
- [Girshick, 2015] Girshick, R. (2015). Fast r-cnn. In *The IEEE International Conference on Computer Vision (ICCV)*.
- [He et al., 2017] He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969.
- [Li et al., 2017] Li, Y., Qi, H., Dai, J., Ji, X., and Wei, Y. (2017). Fully convolutional instance-aware semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2359–2367.
- [Lin et al., 2017] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2017). Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125.
- [Long et al., 2015] Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440.

- [Newell et al., 2016] Newell, A., Yang, K., and Deng, J. (2016). Stacked hourglass networks for human pose estimation. In *European conference on computer vision*, pages 483–499. Springer.
- [Ren et al., 2015] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, pages 91–99.
- [Sun et al., 2019] Sun, K., Xiao, B., Liu, D., and Wang, J. (2019). Deep high-resolution representation learning for human pose estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5693–5703.
- [Targ et al., 2016] Targ, S., Almeida, D., and Lyman, K. (2016). Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*.
- [Tompson et al., 2015] Tompson, J., Goroshin, R., Jain, A., LeCun, Y., and Bregler, C. (2015). Efficient object localization using convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 648–656.
- [Toshev and Szegedy, 2014] Toshev, A. and Szegedy, C. (2014). Deeppose: Human pose estimation via deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1653–1660.
- [Veit et al., 2016] Veit, A., Matera, T., Neumann, L., Matas, J., and Belongie, S. (2016). Coco-text: Dataset and benchmark for text detection and recognition in natural images. *arXiv preprint arXiv:1601.07140*.
- [Wei et al., 2016] Wei, S.-E., Ramakrishna, V., Kanade, T., and Sheikh, Y. (2016). Convolutional pose machines. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 4724–4732.
- [Xiao et al., 2018] Xiao, B., Wu, H., and Wei, Y. (2018). Simple baselines for human pose estimation and tracking. In *Proceedings of the European conference on computer vision (ECCV)*, pages 466–481.